



GBase 8s PL/SQL 手册



GBase 8s PL/SQL手册，南大通用数据技术股份有限公司

GBase 版权所有©2004-2030，保留所有权利

版权声明

本文档所涉及的软件著作权及其他知识产权已依法进行了相关注册、登记，由南大通用数据技术股份有限公司合法拥有，受《中华人民共和国著作权法》、《计算机软件保护条例》、《知识产权保护条例》和相关国际版权条约、法律、法规以及其它知识产权法律和条约的保护。未经授权许可，不得非法使用。

免责声明

本文档包含的南大通用数据技术股份有限公司的版权信息由南大通用数据技术股份有限公司合法拥有，受法律的保护，南大通用数据技术股份有限公司对本文档可能涉及到的非南大通用数据技术股份有限公司的信息不承担任何责任。在法律允许的范围内，您可以查阅，并仅能够在《中华人民共和国著作权法》规定的合法范围内复制和打印本文档。任何单位和个人未经南大通用数据技术股份有限公司书面授权许可，不得使用、修改、再发布本文档的任何部分和内容，否则将视为侵权，南大通用数据技术股份有限公司具有依法追究其责任的权利。

本文档中包含的信息如有更新，恕不另行通知。您对本文档的任何问题，可直接向南大通用数据技术股份有限公司告知或查询。

通讯方式

南大通用数据技术股份有限公司

天津市高新区开华道22号普天创新产业园东塔20-23层

电话：400-013-9696

邮箱：info@gbase.cn

商标声明

GBASE[®]是南大通用数据技术股份有限公司向中华人民共和国国家商标局申请注册的注册商标，注册商标专用权由南大通用数据技术股份有限公司合法拥有，受法律保护。未经南大通用数据技术股份有限公司书面许可，任何单位及个人不得以任何方式或理由对该商标的任何部分进行使用、复制、修改、传播、抄录或与其它产品捆绑使用销售。凡侵犯南大通用数据技术股份有限公司商标权的，南大通用数据技术股份有限公司将依法追究其法律责任。

目 录

PL/SQL语法指南.....	1
1 PL/SQL基础.....	1
1.1 词法单元.....	1
1.1.1 分隔符(Delimiters).....	1
1.1.2 标识符(Identifiers).....	2
1.1.3 注释(Comments).....	6
1.2 常量变量.....	7
1.2.1 声明.....	7
1.2.2 赋值.....	8
1.2.3 初始化.....	10
1.3 表达式.....	11
1.3.1 连接运算符.....	11
1.3.2 操作优先级.....	12
1.3.3 逻辑运算符.....	13
1.3.4 短路计算.....	18
1.3.5 比较运算符.....	19
1.3.6 算术运算符.....	21
1.3.7 布尔表达式.....	22
1.3.8 Case表达式.....	23
2 数据类型.....	26
3 控制语句.....	28
3.1 条件控制.....	28
3.1.1 IF语句.....	28
3.1.2 CASE语句.....	32
3.2 循环控制.....	33
3.2.1 LOOP语句.....	33
3.2.2 FOR LOOP语句.....	34
3.2.3 WHILE LOOP语句.....	40
3.3 顺序控制.....	40
3.3.1 GOTO语句.....	错误！未定义书签。
3.3.2 NULL语句.....	41
4 静态SQL.....	错误！未定义书签。

4.1 、游标.....	46
4.1.1 显式游标.....	46
4.1.2 隐式游标.....	53
4.2 处理结果集.....	56
4.2.1 SELECT INTO语句处理结果集.....	56
4.2.2 FOR LOOP语句处理结果集.....	56
4.2.3 OPEN-FETCH-CLOSE.....	58
4.2.4 处理带有子查询的结果集.....	58
4.3 游标变量.....	59
5 动态SQL.....	68
5.1 使用EXECUTE IMMEDIATE.....	69
5.2 使用游标语法.....	71
6 子程序.....	71
6.1 使用子程序的原因.....	72
6.2 子程序调用.....	72
6.3 子程序属性.....	72
6.4 子程序部分.....	72
6.5 子程序参数.....	74
6.6 子程序调用解析.....	75
6.7 递归子程序.....	75
6.8 子程序副作用.....	76
6.9 SQL语句可以调用的PL/SQL函数.....	77
7 包.....	77
7.1 包的创建.....	77
7.2 包的删除.....	79
7.3 包的引用.....	80
8 DML触发器.....	83
8.1 触发器的创建.....	83
8.2 启用或禁用触发器.....	88
8.3 触发体.....	89
8.4 新、旧行值的引用.....	89
9 自定义类型.....	91
9.1 CREATE TYPE语句创建自定义类型.....	91
9.1.1 对象类型.....	92

9.1.2 嵌套表类型.....	96
9.1.3 可变数组类型.....	97
9.2 CREATE TYPE BODY语句创建自定义类型体.....	98
9.2.1 存储过程实现.....	99
9.2.2 函数实现.....	99
9.3 集合类型在PL/SQL中应用.....	102
9.3.1 嵌套表类型的声明和使用.....	103
9.3.2 可变数组的声明和使用.....	104
10 错误处理.....	108
10.1 PL/SQL运行时的错误处理概述.....	108
10.2 预定义异常.....	109
10.3 自定义异常.....	109
10.4 如何抛出异常.....	113
10.5 异常如何传播.....	114
10.6 重新抛出异常.....	115
10.7 处理抛出的异常.....	116

PL/SQL语法指南

本文描述GBase 8s PL/SQL语法，包括PL/SQL支持的数据类型，变量声明、赋值语法，顺序、选择、循环分支语法，集合、记录对象的使用，静态SQL、动态SQL语法以及错误处理等。

GBase 8s 当前版本所支持的 PL/SQL 语法与Oracle完全一致，原Oracle存储过程可直接迁移，当前版本支持之外的语法，后续会不断完善。

当前版本 PL/SQL 语法需要显式设置环境变量SQLMODE为'ORACLE'后才能生效，默认情况下8s的SQLMODE为'GBASE'，此时不支持 PL/SQL 语法。同时，客户端工具DB-Aaccess 的交互模式也不支持 PL/SQL 语法，可使用DB-Access或GBaseDataStudio工具的SQL编辑器执行 PL/SQL 语法。

1 PL/SQL基础

1.1 词法单元

1.1.1 分隔符(Delimiters)

分隔符用于标记词法元素之间的分割，可以是一个字符，也可以是由多个字符组合组成的，都有特殊的意义，例如“+”。

Delimiter	Meaning
+	Addition operator
:=	Assignment operator
%	Attribute indicator
'	Character string delimiter
.	Component indicator
	Concatenation operator
/	Division operator
(	Expression or list delimiter (begin)
)	Expression or list delimiter (end)
,	Item separator
<<	Label delimiter (begin)
>>	Label delimiter (end)

/*	Multiline comment delimiter (begin)
*/	Multiline comment delimiter (end)
*	Multiplication operator
"	Quoted identifier delimiter
..	Range operator
=	Relational operator (equal)
<>	Relational operator (not equal)
!=	Relational operator (not equal)
~=	Relational operator (not equal)
^=	Relational operator (not equal)
<	Relational operator (less than)
>	Relational operator (greater than)
<=	Relational operator (less than or equal)
>=	Relational operator (greater than or equal)
--	Single-line comment indicator
;	Statement terminator
-	Subtraction or negation operator

1.1.2 标识符(Identifiers)

标识符用于命名PL/SQL语法单元，包括：

- 常量
- 变量
- 异常
- 游标
- 关键字
- 标签
- 保留字
- 类型

标识符中的每个字符都是有意义的，例如lastname和last_name是不同的。必须通过一个或多个空白或者一个标点符号来分隔相邻的标识符。标识符的大小写是不敏感的。例如lastname、Lastname、LASTNAME是相同的。

保留字和关键字

PL/SQL中有一些具有特殊意义的标识符，被称作保留字和关键字。不能使用保留字做为用户定义的标识符。可以用关键字作为用户定义的标识符。

用户自定义保留字

PL/SQL标识符的长度限制，请参照GBase 8s语法。

用户标识符：

- 以字符开头
- 可以包含字母、数字和\$,_,字符
- 不能为保留字

例如以下是可接受的标识符：

```
X
t2
credit_limit
LastName
oracle$number
money$$$tree
try_again_
```

PL/SQL不容许在标识符中使用分隔符，如下所示：

```
mine&yours -- ampersand (&) is not allowed
debit-amount -- hyphen (-) is not allowed
on/off -- slash (/) is not allowed
user id -- space is not allowed
```

PL/SQL对标识符不区分大小写，PL/SQL认为下面命名是相同的。例子

```
lastname
LastName
LASTNAME
```

每个字符都是有意义的。例如，PL/SQL考虑以下是不同的。例子。

```
lastname
last_name
```

为了让PL/SQL程序更加易读，取名要有意义。例如

```
cost_per_thousand    --- 易懂
cpt                  --- 难懂
```

★自定义名称

- 作用域

同一作用域内声明的标识符必须是唯一的。即使是数据类型不同，变量名或参数名也不能相同。

例如：

```
valid_id BOOLEAN ;  
valid_id VARCHAR2(5); -- not allowed duplicate identifier
```

- 大小写敏感

与所有标识符一样，常量、变量和参数的名称是大小写不敏感的

```
zip_code INTEGER ;  
ZIP_CODE INTEGER ; -- same as zip_code
```

- 名称解析

在有二义性的SQL语句中，数据库列的名称优先于局部变量和形参。

例如，如果在WHERE子句中使用使用名字相同的变量和列，SQL将这两个名称都视为具有相同名称的列。下面的DELETE语句会从EMP表中删除所有雇员的信息。而不是只删除KING的雇员。

```
CREATE PROCEDURE p_1_4_3_1 AS  
    ename VARCHAR2(10) := 'KING';  
BEGIN  
    DELETE FROM emp  
    WHERE ename = ename;  
END;  
/
```

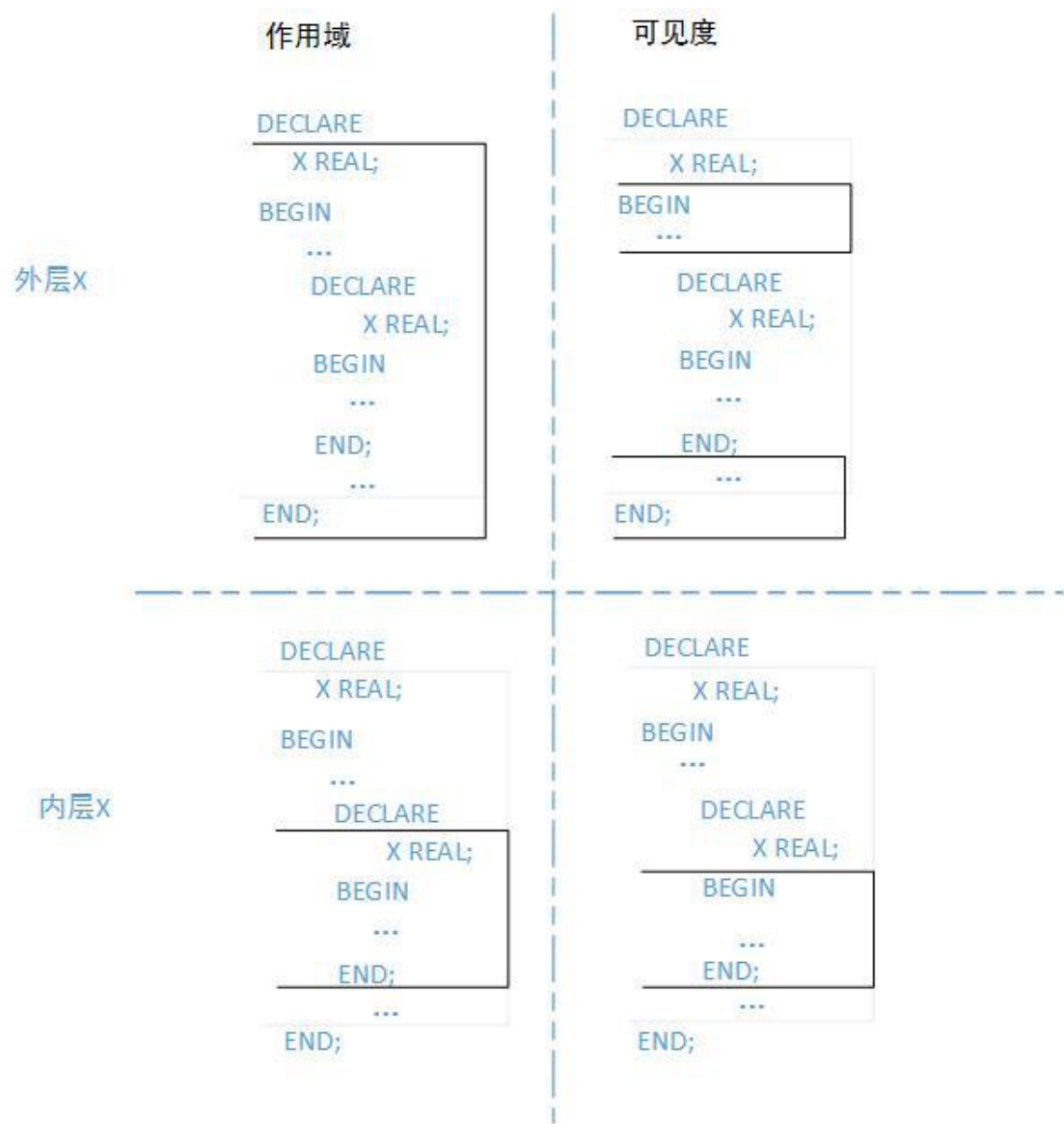
★标识符范围和可见性

对标识符的引用可以通过作用域和可见度来进行解析。

- 作用域：引用标识符的程序单元区域。
- 可见度：一个标识符只有在它的作用域内才能可见。可以在作用域内不使用限定词而直接引用。

声明的标识符对于所在块就是本地的，对于子块就是全局的。如果全局标识符在子块中重新定义，那么两个变量在子块的作用域都是存在的，但只有本地标识符是可见的，如果这时候想引用全局标识符，就要加限定符。虽然不能再同一块中两次声明同一标识符，但可以在不同的块中声明同一标识符。这两个标识符是相互独立的，互不影响。但一个块

中不能引用另一同级别块中的变量。因为它而言，同级块中的标识符既不是本地也不是全局的。如下图：



如下示例展示了几个标识符的范围和可见性。第一个子块重新声明全局标识符a。为了引用全局变量a，第一个子块必须使用外部块的名称来限定它，但是外部块没有名称。因此，第一个子块不能引用全局变量a;它只能引用其局部变量a，因为子块在同一层，第一个子块不能引用d，第二个子块不能引用c。

```
--Outer block:
CREATE PROCEDURE p_1_5 AS
  a CHAR; -- Scope of a (CHAR) begins
  b REAL; -- Scope of b begins
BEGIN
  -- Visible: a (CHAR), b
  -- First sub-block:
```

```
DECLARE
  a INTEGER; -- Scope of a (INTEGER) begins
  c REAL; -- Scope of c begins
BEGIN
  -- Visible: a (INTEGER), b, c
  NULL;
END; -- Scopes of a (INTEGER) and c end

-- Second sub-block:
DECLARE
  d REAL; -- Scope of d begins
BEGIN
  -- Visible: a (CHAR), b, d
  NULL;
END; -- Scope of d ends
-- Visible: a (CHAR), b
END; -- Scopes of a (CHAR) and b end
/
```

★相同范围内的重复标识符

您不能在同一个PL/SQL单元中两次声明相同的标识符。如果您这样做，当您引用重复标识符时，会出现错误，如本例所示。

```
CREATE PROCEDURE p_1_5_3_1 AS
  id BOOLEAN;
  id VARCHAR2(5); -- duplicate identifier
BEGIN
  id := FALSE;
END;
/
669: Variable(id) redeclared.
Error in line 3
Near character position 2
```

1.1.3 注释(Comments)

PL/SQL编译器忽略注释。向程序添加注释可以增强可读性。通常，使用注释来描述每个代码段的用途，也可以将代码通过注释禁用。

★单行注释

--，一直到行为结束，例子如下所示：

```
--select 1 from dual;
```

★多行注释

/*注释内容*/，例子如下所示：

```
/* select 1 from dual;  
select 2 from dual;*/
```

1.2 常量变量

PL/SQL程序将值存储在变量和常量中。当程序执行时变量的值可以更改，但常量的值不能更改。声明为值分配存储空间，指定其数据类型，并命名存储位置，以便后续可以引用它。

1.2.1 声明

★声明变量

变量声明需要指定变量名称和数据类型。对于大多数数据类型，声明变量也可以指定初始值。变量名称必须是有效的用户定义标识符。数据类型可以是任何PL/SQL数据类型。数据类型可以是简单地（SCALAR），也可以是复合的。赋值运算符后面的表达式可以任意复杂，并且可以引用以前初始化的变量。每次执行程序时，都会初始化变量。

例如：

```
CREATE PROCEDURE p_1_2_1_1 AS  
    birthday DATE;  
    emp_count SMALLINT := 0;  
    pi REAL := 3.14159;  
    radius REAL := 1;  
    area REAL := pi * radius*2;  
BEGIN  
    NULL;  
END;  
/
```

★声明常量

声明常量时，需要将关键字CONSTANT放在类型说明符之前。

常量必须在其声明中初始化。这个下面的例子声明命名了实数类型的常量，并指定了一个不可更改的值5000给常数。

```
CREATE PROCEDURE p_1_2_2_1 AS  
    credit_limit CONSTANT REAL := 5000.00;  
    max_days_in_year CONSTANT INTEGER := 366;  
    urban_legend CONSTANT BOOLEAN := FALSE;
```

```
BEGIN
  NULL;
END;
/
```

注意：不允许向前引用

下面的声明是不合法的

```
maxi INTEGER := 2 * mini; -- not allowed
mini INTEGER := 15;
```

下面的声明也是不允许的

```
j, k SMALLINT ; -- not allowed
```

1.2.2 赋值

有3中方式赋值：

- 使用赋值语句将表达式赋值给变量
- 使用SELECT INTO 或 FETCH语句赋值给变量
- 将变量传递给子程序的OUT或IN OUT 参数在子程序里赋值。

声明变量时，可以将默认值赋给该变量或声明后使用赋值语句。赋值运算符（:=）后面的表达式可以任意复杂。默认情况下，变量初始化为空。除非显式初始化变量，否则其值为未知的。

★用赋值语句给变量赋值

这个例子声明了几个变量(为一些变量指定初始值)，然后使用赋值语句将表达式的值赋给它们。

```
CREATE PROCEDURE p_1_6_1 AS -- You can assign initial values here
  wages          NUMBER;
  hours_worked   NUMBER := 40;
  hourly_salary  NUMBER := 22.50;
  bonus          NUMBER := 150;
  country        VARCHAR2(128);
  counter        NUMBER := 0;
  done           BOOLEAN;
  valid_id       BOOLEAN;
  emp_rec1       employees%ROWTYPE;
  emp_rec2       employees%ROWTYPE;
  TYPE commissions IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  comm_tab       commissions;
```

```
BEGIN    -- You can assign values here too
    wages := (hours_worked * hourly_salary) + bonus;
    country := 'France';
    country := UPPER('Canada');
    done := (counter > 100);
    valid_id := TRUE;
    emp_rec1.first_name := 'Antonio';
    emp_rec1.last_name := 'Ortiz';
    emp_rec1 := emp_rec2;
    comm_tab(5) := 20000 * 0.15;
END;
/
```

★用SELECT INTO语句给变量赋值

本例使用SELECT INTO语句将employee_id为100的雇员的工资的10%赋值给变量 bonus。

```
CREATE PROCEDURE p_1_6_2 AS
    bonus NUMBER(8,2);
BEGIN
    SELECT salary * 0.10 INTO bonus
    FROM employees
    WHERE employee_id = 100;
    DBMS_OUTPUT.PUT_LINE('bonus = ' || TO_CHAR(bonus));
END;
/
--Result:
--bonus = 2400
```

★IN OUT子程序参数给变量赋值

这个示例将变量new_sal传递给过程adjust_salary。该过程将一个值赋给相应的形式参数sal。由于sal是一个 IN OUT 参数，变量new_sal在过程结束后保留了赋值。

```
CREATE PROCEDURE adjust_salary (
    emp        NUMBER,
    sal IN OUT  NUMBER,
    adjustment  NUMBER
) IS
BEGIN
    sal := sal + adjustment;
END;
/

CREATE PROCEDURE p_1_6_3 AS
```

```
emp_salary NUMBER(8,2);
BEGIN
  SELECT salary INTO emp_salary
  FROM employees
  WHERE employee_id = 100;
  DBMS_OUTPUT.PUT_LINE
  ('Before invoking procedure, emp_salary: ' || emp_salary);
  adjust_salary (100, emp_salary, 1000);
  DBMS_OUTPUT.PUT_LINE
  ('After invoking procedure, emp_salary: ' || emp_salary);
END;
/

--Result:
--Before invoking procedure, emp_salary: 24000
--After invoking procedure, emp_salary: 25000
```

★给BOOLEAN变量赋值

只有值TRUE、FALSE和NULL可以分配给布尔变量。这个示例布尔变量done默认初始化为NULL，后将其赋值为FALSE。

```
CREATE PROCEDURE p_1_6_4 AS
done BOOLEAN;          -- Initial value is NULL by default
counter NUMBER := 0;
BEGIN
  done := FALSE;        -- Assign literal value
  WHILE done != TRUE    -- Compare to literal value
  LOOP
    counter := counter + 1;
    done := (counter > 500); -- Assign value of BOOLEAN expression
  END LOOP;
END;
/
```

1.2.3 初始化

在变量的声明中初始化时可选的。除非指定NO NULL。在常量中初始化时必须的。常量不指定初始值，默认就为NULL。使用:=和DEFAULT关键字进行初始化。后边跟表达式，表达式可以包含前面已经声明的常量或已经初始化的变量。

```
CREATE PROCEDURE p_1_2_3_1 AS
  hours_worked INTEGER := 40;
  employee_count INTEGER := 0;
```



```
pi CONSTANT REAL := 3.14159;
radius REAL := 1;
area REAL := (pi * radius*2);
BEGIN
  NULL;
END;
/
```

```
CREATE PROCEDURE p_1_2_3_2 AS
  counter INTEGER;          -- initial value is NULL by default
BEGIN
  counter := counter + 1;    -- NULL + 1 is still NULL
  IF counter IS NULL THEN
    DBMS_OUTPUT.PUT_LINE('counter is NULL.');
```

```
CREATE PROCEDURE p_1_2_3_3 AS
  blood_type CHAR DEFAULT 'O';      -- Same as blood_type CHAR := 'O';
  hours_worked INTEGER DEFAULT 40;  -- Typical value
  employee_count INTEGER := 0;      -- No typical value
BEGIN
  NULL;
END;
/
```

1.3 表达式

一个表达式始终返回唯一值。表达式有：

- 单个常量或变量
- 一个操作符和单个操作数
- 二元操作符和2个操作数

操作数可以是：常量、变量、文字等。操作数的数据类型决定了表达式的数据类型。

1.3.1 连接运算符

```
CREATE PROCEDURE p_1_7_1_1 AS
  x VARCHAR2(4) := 'suit';
  y VARCHAR2(4) := 'case';
BEGIN
  DBMS_OUTPUT.PUT_LINE (x || y);
END;
/
--Result:
--suitcase
```

1.3.2 操作优先级

运算是一元操作符操作及单个操作数或二元操作符及其两个操作数。表达式中的操作按照操作符的优先级的顺序计算。表达式中的操作按优先顺序计算。显示从最高到最低的运算符优先级。具有同等优先级的运算符。

Operator	Operation
+, -	identity, negation
*, /	multiplication, division
+, -,	addition, subtraction,concatenation
=, <, >, <=, >=, <>, !=, ~=, ^=, IS NULL, LIKE, BETWEEN, IN	comparison
NOT	negation
AND	conjunction
OR	inclusion

★用括号控制评价顺序

```
CREATE PROCEDURE p_1_7_2_1 AS
  a INTEGER := 1+2*2;
  b INTEGER := (1+2)*2;
BEGIN
  DBMS_OUTPUT.PUT_LINE('a = ' || TO_CHAR(a));
  DBMS_OUTPUT.PUT_LINE('b = ' || TO_CHAR(b));
END;
/
--Result:
--a = 5
--b = 6
```

★嵌套括号的表达式

在本例中，首先计算操作(1+2)和(3+4)，分别生成值3和7。接下来，对操作3*7进行评估，生成结果21。最后，计算操作21/7，生成最终值3。

```
CREATE PROCEDURE p_1_7_2_2 AS
  a INTEGER := ((1+2)*(3+4))/7;
BEGIN
  DBMS_OUTPUT.PUT_LINE('a = ' || TO_CHAR(a));
END;
/
--Result:
--a = 3
```

★运算符的优先级

这个例子展示了运算符优先级和括号在几个更复杂的表达式中的效果。

```
CREATE PROCEDURE p_1_7_2_3 AS
  salary      NUMBER := 60000;
  commission  NUMBER := 0.10;
BEGIN
  -- Division has higher precedence than addition:
  DBMS_OUTPUT.PUT_LINE('5 + 12 / 4 = ' || TO_CHAR(5 + 12 / 4));
  DBMS_OUTPUT.PUT_LINE('12 / 4 + 5 = ' || TO_CHAR(12 / 4 + 5));
  -- Parentheses override default operator precedence:
  DBMS_OUTPUT.PUT_LINE('8 + 6 / 2 = ' || TO_CHAR(8 + 6 / 2));
  DBMS_OUTPUT.PUT_LINE('(8 + 6) / 2 = ' || TO_CHAR((8 + 6) / 2));
  -- Most deeply nested operation is evaluated first:
  DBMS_OUTPUT.PUT_LINE('100 + (20 / 5 + (7 - 3)) = ' || TO_CHAR(100 + (20 / 5 + (7 - 3))));
  -- Parentheses, even when unnecessary, improve readability:
  DBMS_OUTPUT.PUT_LINE('(salary * 0.05) + (commission * 0.25) = '
    || TO_CHAR((salary * 0.05) + (commission * 0.25)));
  DBMS_OUTPUT.PUT_LINE('salary * 0.05 + commission * 0.25 = '
    || TO_CHAR(salary * 0.05 + commission * 0.25));
END;
/
--Result:
--5 + 12 / 4 = 8
--12 / 4 + 5 = 8
--8 + 6 / 2 = 11
--(8 + 6) / 2 = 7
--100 + (20 / 5 + (7 - 3)) = 108
--(salary * 0.05) + (commission * 0.25) = 3000.025
--salary * 0.05 + commission * 0.25 = 3000.025
```

1.3.3 逻辑运算符

逻辑运算符AND、OR和NOT遵循下表所示的三态逻辑。但是要避免使用NULL的异常情况。

x	y	x AND y	x OR y	NOT x
TRUE	TRUE	TRUE	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	FALSE
TRUE	NULL	NULL	TRUE	FALSE
FALSE	TRUE	FALSE	TRUE	TRUE
FALSE	FALSE	FALSE	FALSE	TRUE
FALSE	NULL	FALSE	NULL	TRUE
NULL	TRUE	NULL	TRUE	NULL
NULL	FALSE	FALSE	NULL	NULL
NULL	NULL	NULL	NULL	NULL

AND和OR是二进制运算符；NOT是一元运算符。当且仅当两个操作数为TURE，AND返回TURE；当有一个为TURE，OR返回TURE;NOT返回操作数的相反值. NOT NULL还是NULL。因为NULL本身就是一个不确定的值。

★程序输出布尔变量

这个例子创建一个print_boolean程序，它输出一个 BOOLEAN 变量的值。该过程使用“IS [NOT] NULL Operator”。

```
CREATE OR REPLACE PROCEDURE print_boolean (  
  b_name VARCHAR2, b_value BOOLEAN) AUTHID DEFINER IS  
BEGIN  
  IF b_value IS NULL THEN  
    DBMS_OUTPUT.PUT_LINE (b_name || ' = NULL');  
  ELSIF b_value = TRUE THEN  
    DBMS_OUTPUT.PUT_LINE (b_name || ' = TRUE');  
  ELSE  
    DBMS_OUTPUT.PUT_LINE (b_name || ' = FALSE');  
  END IF;  
END;  
/
```

★与操作

如果两个操作数都为真，则返回TRUE。

x	y	x AND y	x OR y	NOT x
TRUE	TRUE	TRUE	TRUE	FALSE

TRUE	FALSE	FALSE	TRUE	FALSE
TRUE	NULL	NULL	TRUE	FALSE
FALSE	TRUE	FALSE	TRUE	TRUE
FALSE	FALSE	FALSE	FALSE	TRUE
FALSE	NULL	FALSE	NULL	TRUE
NULL	TRUE	NULL	TRUE	NULL
NULL	FALSE	FALSE	NULL	NULL
NULL	NULL	NULL	NULL	NULL

```
CREATE OR REPLACE PROCEDURE p_1_7_3_2
AS
  x boolean :=true;
  y boolean :=false;
BEGIN
  if x and y then
    DBMS_OUTPUT.PUT_LINE('true');
  else
    DBMS_OUTPUT.PUT_LINE('x and y');
  end if;
END;
/
--Result:
-- x and y
```

★或运算

或返回TRUE，如果操作数是正确的。

```
CREATE OR REPLACE PROCEDURE p_1_7_3_3
AS
  x boolean :=true;
  y boolean :=false;
BEGIN
  if x or y then
    DBMS_OUTPUT.PUT_LINE('x or y');
  end if;
END;
/
--Result:
-- x or y
```

★求反算符

NOT 返回其操作数的相反值，除非操作数为 NULL。NOT NULL 返回 NULL，因为 NULL 是一个不确定值。

```
CREATE OR REPLACE PROCEDURE p_1_7_3_4
AS
  x boolean := true;
BEGIN
  if not x then
    DBMS_OUTPUT.PUT_LINE('true');
  else
    DBMS_OUTPUT.PUT_LINE('not x');
  end if;
END;
/
--Result:
--not x
```

★不相等比较中的NULL值

在本例中，您可能期望运行的语句序列，因为x和y看起来不相等。但是，空值是不确定的，不论x = y是否未知。因此，IF条件为NULL，那么语句的序列就被忽略了。

```
CREATE PROCEDURE p_1_7_3_5
AS
  x NUMBER := 5;
  y NUMBER := NULL;
BEGIN
  IF x != y THEN -- yields NULL, not TRUE
    DBMS_OUTPUT.PUT_LINE('x != y'); -- not run
  ELSIF x = y THEN -- also yields NULL
    DBMS_OUTPUT.PUT_LINE('x = y');
  ELSE
    DBMS_OUTPUT.PUT_LINE
      ('Can''t tell if x and y are equal or not.');
```

★相等比较中的NULL值

在本例中，您可能期望运行的语句序列，因为a和b看起来是相等的。但是，同样的，这是未知的，所以IF条件产生了NULL，那么语句的序列就被忽略了。

```
CREATE PROCEDURE p_1_7_3_6
AS
  a NUMBER := NULL;
  b NUMBER := NULL;
BEGIN
  IF a = b THEN -- yields NULL, not TRUE
    DBMS_OUTPUT.PUT_LINE('a = b'); -- not run
  ELSIF a != b THEN -- yields NULL, not TRUE
    DBMS_OUTPUT.PUT_LINE('a != b'); -- not run
  ELSE
    DBMS_OUTPUT.PUT_LINE('Can''t tell if two NULLs are equal');
  END IF;
END;
/
--Result:
--Can't tell if two NULLs are equal
```

★NOT NULL等于NULL

在本例中，两个IF语句似乎是等价的。但是，如果x或y有一个是NULL，那么第一个IF语句将y的值赋值给high，而第二个if语句将x的值赋值为high。

```
CREATE PROCEDURE p_1_7_3_7 AS
  x INTEGER := 2;
  Y INTEGER := 5;
  high INTEGER;
BEGIN
  IF (x > y) -- If x or y is NULL, then (x > y) is NULL
  THEN high := x; -- run if (x > y) is TRUE
  ELSE high := y; -- run if (x > y) is FALSE or NULL
  END IF;
  IF NOT (x > y) -- If x or y is NULL, then NOT (x > y) is NULL
  THEN high := y; -- run if NOT (x > y) is TRUE
  ELSE high := x; -- run if NOT (x > y) is FALSE or NULL
  END IF;
END;
/
```

★更改逻辑运算符的运算顺序

第三次调用和第一次调用在逻辑上等价于第三次调用中的圆括号，只提高了可读性。第二个调用中的括号将更改运算顺序。

```
CREATE OR REPLACE PROCEDURE p_1_7_3_8
AS
```

```
x boolean :=true;
y boolean :=false;
BEGIN
  if NOT x AND y then
    DBMS_OUTPUT.PUT_LINE('NOT x AND y is true');
  else
    DBMS_OUTPUT.PUT_LINE('NOT x AND y is false');
  end if;
  if NOT (x AND y) then
    DBMS_OUTPUT.PUT_LINE('NOT (x AND y) is true');
  else
    DBMS_OUTPUT.PUT_LINE('NOT (x AND y) is false');
  end if;
  if (NOT x) AND y then
    DBMS_OUTPUT.PUT_LINE('(NOT x) AND y is true');
  else
    DBMS_OUTPUT.PUT_LINE('(NOT x) AND y is false');
  end if;
END;
/
--Result:
--NOT x AND y is false
--NOT (x AND y) is true
--(NOT x) AND y is false
```

1.3.4 短路计算

当计算一个逻辑表达式的时候，PL/SQL使用短路计算。一旦能够确认表达式的值，则就不进行后续计算了。

```
CREATE PROCEDURE p_1_7_4
AS
  on_hand INTEGER := 0;
  on_order INTEGER := 100;
BEGIN
  -- Does not cause divide-by-zero error;
  -- evaluation stops after first expression
  IF (on_hand = 0) OR ((on_order / on_hand) < 5) THEN
    DBMS_OUTPUT.PUT_LINE('On hand quantity is zero.');
```



```
--Result:
--On hand quantity is zero.
```

1.3.5 比较运算符

比较操作符用于一个表达式和另外一个进行比较。结果为TRUE、FALSE、NULL。当一个表达式为NULL，比较的结果也为NULL。

比较操作符

Operator	Meaning
=	equal to
<>, !=, ~=, ^=	not equal to
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to

★关系运算符的表达式

这个例子输出使用关系运算符比较算术值的表达式的值。

```
CREATE OR REPLACE PROCEDURE p_1_7_5_1
AS
BEGIN
  if (2 + 2 ^= 4) then
    DBMS_OUTPUT.PUT_LINE('(2 + 2 ^= 4) is true');
  else
    DBMS_OUTPUT.PUT_LINE('(2 + 2 ^= 4) is false');
  end if;
END;
/
--Result:
--(2 + 2 ^= 4) is false
```

★LIKE运算表达式

字符串'Johnson'匹配模式'J%s_n'，但不是'J%s_N'，就像这个例子所显示的那样。

```
CREATE PROCEDURE compare(value VARCHAR2, pattern VARCHAR2)
IS
BEGIN
  IF value LIKE pattern THEN
    DBMS_OUTPUT.PUT_LINE ('TRUE');
  ELSE
    DBMS_OUTPUT.PUT_LINE ('FALSE');
  END IF;
```

```
END;
/
call compare('Johnson', 'J%s_n');
call compare('Johnson', 'J%S_N');
--Result:
--TRUE
--FALSE
```

★转义字符模式

这个例子使用反斜杠作为转义字符，因此字符串中的百分号不作为通配符。

```
CREATE PROCEDURE half_off (sale_sign VARCHAR2)
IS
BEGIN
    IF sale_sign LIKE '50\% off!' ESCAPE '\' THEN
        DBMS_OUTPUT.PUT_LINE ('TRUE');
    ELSE
        DBMS_OUTPUT.PUT_LINE ('FALSE');
    END IF;
END;
/
call half_off('Going out of business!');
call half_off('50% off!');
--Result:
--FALSE
--TRUE
```

★BETWEEN运算表达式

这个例子输出包含BETWEEN运算符的表达式值。

```
CREATE OR REPLACE PROCEDURE p_1_7_5_4
AS
BEGIN
    if 2 BETWEEN 1 AND 3 then
        DBMS_OUTPUT.PUT_LINE('2 BETWEEN 1 AND 3 is true');
    else
        DBMS_OUTPUT.PUT_LINE('2 BETWEEN 1 AND 3 is false');
    end if;
END;
/
--Result:
--2 BETWEEN 1 AND 3 is true
```

★IN运算表达式

这个例子输出包含IN运算符的表达式值。

```
CREATE OR REPLACE PROCEDURE p_1_7_5_5
AS
    letter VARCHAR2(1) := 'm';
BEGIN
    if letter IN ('a', 'b', 'c') then
        DBMS_OUTPUT.PUT_LINE('letter IN (a, b, c) is true');
    else
        DBMS_OUTPUT.PUT_LINE('letter IN (a, b, c) is false');
    end if;
END;
/
--Result:
- letter IN (a, b, c) is false
```

★有NULL的IN运算集

这个示例显示了当集合包含NULL值时发生的情况。

```
CREATE OR REPLACE PROCEDURE p_1_7_5_6
AS
    a INTEGER; -- Initialized to NULL by default
    b INTEGER := 10;
    c INTEGER := 100;
BEGIN
    if (100 IN (a, b, c)) then
        DBMS_OUTPUT.PUT_LINE('(100 IN (a, b, c)) is true');
    else
        DBMS_OUTPUT.PUT_LINE('(100 IN (a, b, c)) is false');
    end if;
END;
/
--Result:
--100 IN (a, b, c) is true
```

1.3.6 算术运算符

★MOD运算符

MOD运算符用于返回n1 除以 n2 的余数。如果 n2 为 0，则返回 n1。



参数说明：

元素	描述	限制	语
----	----	----	---

			法
<i>n1</i>	被除数	可以是任何数值类型或者可以隐式转换成数值类型的其他数据类型。	表达式
<i>n2</i>	除数	可以是任何数值类型或者可以隐式转换成数值类型的其他数据类型。	表达式

例如，函数中使用mod运算符：

```
SQL>create or replace function f1(i int)
return int
is
result int;
begin
    result :=i mod 3;
    return result;
end ;
/
```

1.3.7 布尔表达式

在SQL语句中，布尔表达式允许您在表中指定受声明影响。在过程语句中，布尔表达式是一个总能返回TURE或FLASE或NULL的表达式。一个简单的布尔表达式可以有布尔值、常量、变量组成。常见形式如下：

```
NOT boolean_expression
boolean_expression relational_operator boolean_expression
boolean_expression { AND | OR } boolean_expression
```

通常，布尔表达式由逻辑运算符AND、OR和NOT连接。布尔表达式总是产生TRUE、FALSE或NULL。

在这个例子中，循环中的条件是等价的。

```
CREATE PROCEDURE p_1_7_6
AS
    done BOOLEAN;
BEGIN
    -- These WHILE loops are equivalent
    done := FALSE;
    WHILE done = FALSE LOOP
        done := TRUE;
    END LOOP;
    done := FALSE;
```

```
WHILE NOT (done = TRUE) LOOP
    done := TRUE;
END LOOP;
done := FALSE;
WHILE NOT done
    LOOP
    done := TRUE;
    END LOOP;
END;
/
```

1.3.8 Case表达式

语法

```
CASE selector
WHEN selector_value_1 THEN result_1
WHEN selector_value_2 THEN result_2
...
WHEN selector_value_n THEN result_n
[ ELSE
else_result ]
END
```

一个CASE表达式从一个或多个方案中选择一个。一旦有一个条件满足，剩下的分支就不执行了。

★简单的CASE表达

这个示例将一个简单CASE表达式的值赋给变量appraisal，selector值是 grade。

```
CREATE PROCEDURE p_1_7_7_1
AS
    grade CHAR(1) := 'B';
    appraisal VARCHAR2(20);
BEGIN
    appraisal :=
        CASE grade
            WHEN 'A' THEN 'Excellent'
            WHEN 'B' THEN 'Very Good'
            WHEN 'C' THEN 'Good'
            WHEN 'D' THEN 'Fair'
            WHEN 'F' THEN 'Poor'
            ELSE 'No such grade'
        END;
END;
```

```
DBMS_OUTPUT.PUT_LINE ('Grade ' || grade || ' is ' || appraisal);
END;
/
--Result:
--Grade B is Very Good
```

★WHEN NULL条件下的简单的CASE表达式

如果selector值为NULL，则不能在WHEN NULL条件下被匹配，相反，可以在WHEN boolean_expression IS NULL条件下使用CASE表达式搜索。

```
CREATE PROCEDURE p_1_7_7_2
AS
    grade CHAR(1); -- NULL by default
    appraisal VARCHAR2(20);
BEGIN
    appraisal :=
        CASE grade
            WHEN NULL THEN 'No grade assigned'
            WHEN 'A' THEN 'Excellent'
            WHEN 'B' THEN 'Very Good'
            WHEN 'C' THEN 'Good'
            WHEN 'D' THEN 'Fair'
            WHEN 'F' THEN 'Poor'
            ELSE 'No such grade'
        END;
    DBMS_OUTPUT.PUT_LINE ('Grade ' || grade || ' is ' || appraisal);
END;
/
--Result:
--Grade is No such grade
```

★搜寻CASE表达式

语法

```
CASE
WHEN boolean_expression_1 THEN result_1
WHEN boolean_expression_2 THEN result_2
...
WHEN boolean_expression_n THEN result_n
[ ELSE
else_result ]
END]
```

这个示例将搜寻CASE表达式的值赋给变量appraisal。

```
CREATE FUNCTION attends_this_school (id NUMBER)
RETURN BOOLEAN IS
a BOOLEAN:=TRUE;
BEGIN
    RETURN a;
END;
/
CREATE PROCEDURE p_1_7_7_3 AS  grade CHAR(1) := 'B';
    appraisal VARCHAR2(120);
    id NUMBER := 8429862;
    attendance NUMBER := 150;
    min_days CONSTANT NUMBER := 200;
BEGIN
    appraisal :=
        CASE
            WHEN attends_this_school(id) = FALSE
                THEN 'Student not enrolled'
            WHEN grade = 'F' OR attendance < min_days
                THEN 'Poor (poor performance or bad attendance)'
            WHEN grade = 'A' THEN 'Excellent'
            WHEN grade = 'B' THEN 'Very Good'
            WHEN grade = 'C' THEN 'Good'
            WHEN grade = 'D' THEN 'Fair'
            ELSE 'No such grade'
        END;
    DBMS_OUTPUT.PUT_LINE('Result for student ' || id || ' is ' || appraisal);
END;
/
--Result:
--Result for student 8429862 is Poor (poor performance or bad attendance)
```

★WHEN ... IS NULL条件下的搜寻case语句

这个示例使用一个搜索的CASE表达式来解决例2.2.51中的问题。

```
CREATE PROCEDURE p_1_7_7_4
AS
    grade CHAR(1); -- NULL by default
    appraisal VARCHAR2(20);
BEGIN
    appraisal :=
        CASE
            WHEN grade IS NULL THEN 'No grade assigned'
            WHEN grade = 'A' THEN 'Excellent'
            WHEN grade = 'B' THEN 'Very Good'
```

```
        WHEN grade = 'C' THEN 'Good'
        WHEN grade = 'D' THEN 'Fair'
        WHEN grade = 'F' THEN 'Poor'
        ELSE 'No such grade'
    END;

    DBMS_OUTPUT.PUT_LINE ('Grade ' || grade || ' is ' || appraisal);
END;
/
--Result:
--Grade is No grade assigned
```

- ★在比较和条件语句中处理NULL值
- 涉及空值的比较总是产生空值
 - 将逻辑运算符NOT应用于空值会产生空值
 - 在条件控制语句中，如果条件为空，则不执行语句序列
 - 如果简单CASE语句或CASE表达式中的表达式产生NULL，则无法通过使用WHEN NULL进行匹配。相反，使用搜索的CASE语法当表达式为空时

2 数据类型

GBase 8s支持的数据类型如下：

类型分类	数据类型	备注
数值型	NUMBER	
	DECIMAL/DEC	
	INTEGER/INT	
	DOUBLE PRECISION	
	BIGINT	
	INT8	
	REAL	
	SMALLINT	
字符型	CHAR	
	NCHAR	
	VARCHAR	
	NVARCHAR	
布尔型	BOOLEAN	
时间类型	DATE	
	TIMESTAMP[(precision)]	
	INTERVAL YEAR[(precision)] TO MONTH	

CHAR和VARCHAR空白填充差异

在本例中，CHAR变量和VARCHAR变量的最大大小为10个字符。每个变量接收一个5字符的值，其中一个为空格。

赋值给CHAR变量的值是空白填充到10个字符，您无法判断结果值中的6个尾随空格中哪一个是原始值。分配给VARCHAR变量的值没有更改，您可以看到它有一个末尾空格。

```
CREATE OR REPLACE PROCEDURE p_2_1 AS
    first_name CHAR(10);
    last_name VARCHAR(10);
BEGIN
    first_name := 'John ';
    last_name := 'Chen ';
    DBMS_OUTPUT.PUT_LINE('*' || first_name || '*');
    DBMS_OUTPUT.PUT_LINE('*' || last_name || '*');
END;
/
--Result:
--*John      *
--*Chen      *
```

输出BOOLEAN值

在本例中，如果参数的值为NULL，该过程接受一个 BOOLEAN参数并使用一个CASE语句来输出Unknown值。如果是TRUE，则为Yes，如果为FALSE，则为No。

```
CREATE OR REPLACE PROCEDURE print_boolean(b boolean)
AS
BEGIN
    DBMS_OUTPUT.PUT_LINE (
        CASE
            WHEN b IS NULL THEN 'Unknown'
            WHEN b THEN 'Yes'
            WHEN NOT b THEN 'No'
        END
    );
END;
/
call print_boolean(TRUE);
call print_boolean(FALSE);
call print_boolean(NULL);

--Result:
--Yes
--No
--Unknown
```

SQL语句通过BOOLEAN函数调用PL/SQL函数

在本例中，SQL语句调用具有BOOLEAN参数的PL/SQL函数。

```
CREATE OR REPLACE FUNCTION f(x BOOLEAN, y INT)
RETURN INT AS
BEGIN
    IF x THEN
        RETURN y;
    ELSE
        RETURN 2*y;
    END IF;
END;
/
CREATE OR REPLACE PROCEDURE p_2_3 AS
    name varchar(30);
    b BOOLEAN := TRUE;
BEGIN
    SELECT last_name INTO name
    FROM employees
    WHERE employee_id = f(b, 100);
    DBMS_OUTPUT.PUT_LINE(name);
    b := FALSE;
    SELECT last_name INTO name
    FROM employees
    WHERE employee_id = f(b, 100);
    DBMS_OUTPUT.PUT_LINE(name);
END;
/
--Result:
--King
--Whalen
```

3 控制语句

3.1 条件控制

3.1.1 IF语句

当PL/SQL执行到IF控制分支的语句时，首先判断条件，根据条件表达式的值选择相应的语句执行（放弃另一部分语句的执行）。使用范围：

- 1.支持控制语句的嵌套
- 2.分支结构包括单分支、双分支和多分支三种形式。

★IF THEN语句

在本例中，THEN到END IF之间的语句仅在sales大于quota+200的情况下执行。

```
CREATE PROCEDURE p_3_1_1 (sales NUMBER, quota NUMBER, emp_id NUMBER) IS
    bonus NUMBER := 0;
    updated VARCHAR2(3) := 'No';
BEGIN
    IF sales > (quota + 200) THEN
        bonus := (sales - quota)/4;
        UPDATE employees
        SET salary = salary + bonus
        WHERE employee_id = emp_id;
        updated := 'Yes';
    END IF;
    DBMS_OUTPUT.PUT_LINE ('Table updated? ' || updated || ', ' ||
        'bonus = ' || bonus || '.');
END p;
/
call p(10100, 10000, 120);
call p(10500, 10000, 121);

--Result:
--Table updated? No, bonus = 0.
--Table updated? Yes, bonus = 125.
```

★IF THEN ELSE语句

在本例中，如果当且仅当sale的值大于quota+200时，才会运行THEN到ELSE之间的语句；否则，运行ELSE和END IF之间的语句。

```
CREATE PROCEDURE p_3_1_2 (sales NUMBER, quota NUMBER, emp_id NUMBER) IS
    bonus NUMBER := 0;
BEGIN
    IF sales > (quota + 200) THEN
        bonus := (sales - quota)/4;
    ELSE
        bonus := 50;
    END IF;
    DBMS_OUTPUT.PUT_LINE('bonus = ' || bonus);
```

```
UPDATE employees
SET salary = salary + bonus
WHERE employee_id = emp_id;
END p;
/
call p(10100, 10000, 120);
call p(10500, 10000, 121);

--Result:
--bonus = 50
--bonus = 125
```

★嵌套IF THEN ELSE语句

```
CREATE PROCEDURE p_3_1_3 (sales NUMBER, quota NUMBER, emp_id NUMBER) IS
bonus NUMBER := 0;
BEGIN
IF sales > (quota + 200) THEN
bonus := (sales - quota)/4;
ELSE
IF sales > quota THEN
bonus := 50;
ELSE
bonus := 0;
END IF;
END IF;
DBMS_OUTPUT.PUT_LINE('bonus = ' || bonus);
UPDATE employees
SET salary = salary + bonus
WHERE employee_id = emp_id;
END p;
/
call p(10100, 10000, 120);
call p(10500, 10000, 121);
call p(9500, 10000, 122);

--Result:
--bonus = 50
--bonus = 125
--bonus = 0
```

★IF THEN ELSIF语句

在这个例子中，当sales大于50000时，第一个和第二个条件是正确的。但是，因为第一个条件是正确的，所以分配了bonus值1500，而第二个条件没有经过测试。在bonus被分配了价值之后,控制传递到DBMS_OUTPUT.PUT_LINE调用。

```
CREATE PROCEDURE p_3_1_4 (sales NUMBER) IS
    bonus NUMBER := 0;
BEGIN
    IF sales > 50000 THEN
        bonus := 1500;
    ELSIF sales > 35000 THEN
        bonus := 500;
    ELSE
        bonus := 100;
    END IF;
    DBMS_OUTPUT.PUT_LINE ('Sales = ' || sales || ', bonus = ' || bonus || '.');
END p;
/
call p_3_1_4 (55000);
call p_3_1_4 (40000);
call p_3_1_4 (30000);

--Result:
--Sales = 55000, bonus = 1500.
--Sales = 40000, bonus = 500.
--Sales = 30000, bonus = 100.
```

★IF THEN ELSIF语句模拟简单的CASE语句

这个例子使用IF THEN ELSIF语句和许多ELSIF子句来比较单个值和许多可能的值。简单的CASE语句更清楚。

```
CREATE PROCEDURE p_3_1_5 AS
    grade CHAR(1);
BEGIN
    grade := 'B';
    IF grade = 'A' THEN
        DBMS_OUTPUT.PUT_LINE('Excellent');
    ELSIF grade = 'B' THEN
        DBMS_OUTPUT.PUT_LINE('Very Good');
    ELSIF grade = 'C' THEN
        DBMS_OUTPUT.PUT_LINE('Good');
    ELSIF grade = 'D' THEN
        DBMS_OUTPUT.PUT_LINE('Fair');
    ELSIF grade = 'F' THEN
        DBMS_OUTPUT.PUT_LINE('Poor');
```

```
ELSE
    DBMS_OUTPUT.PUT_LINE('No such grade');
END IF;
END;
/
--Result:
--Very Good
```

3.1.2 CASE语句

case语句可以使程序结构比较清晰。分为简单case语句和搜索式case语句两种。使用范围:

1. 当只有一个搜索条件时, 计算选择简单 case 语句。Case 关键字后会跟 selector 表达式, selector 只计算一次, 并和表达式的值进行比较。如果两者匹配, 对应的 case 语句的 statement 会被执行。
2. 当有多个搜索条件时, 需要选择搜索式 case 语句。注意, case 关键字后面没有 selector 表达式, 当特定条件满足时, 会执行该条件相关的 statement。

★简单的CASE语句

```
CREATE PROCEDURE p_3_1_6 AS
    grade CHAR(1);
BEGIN
    grade := 'B';
    CASE grade
        WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent');
        WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('Very Good');
        WHEN 'C' THEN DBMS_OUTPUT.PUT_LINE('Good');
        WHEN 'D' THEN DBMS_OUTPUT.PUT_LINE('Fair');
        WHEN 'F' THEN DBMS_OUTPUT.PUT_LINE('Poor');
    ELSE
        DBMS_OUTPUT.PUT_LINE('No such grade');
    END CASE;
END;
/
--Result:
--Very Good
```

★搜索CASE语句

```
CREATE PROCEDURE p_3_1_7 AS
    grade CHAR(1);
BEGIN
    grade := 'B';
```

```
CASE
  WHEN grade = 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent');
  WHEN grade = 'B' THEN DBMS_OUTPUT.PUT_LINE('Very Good');
  WHEN grade = 'C' THEN DBMS_OUTPUT.PUT_LINE('Good');
  WHEN grade = 'D' THEN DBMS_OUTPUT.PUT_LINE('Fair');
  WHEN grade = 'F' THEN DBMS_OUTPUT.PUT_LINE('Poor');
  ELSE
    DBMS_OUTPUT.PUT_LINE('No such grade');
  END CASE;
END;
/
--Result:
--Very Good
```

3.2 循环控制

PL/SQL用循环结构可以实现有规律的重复计算处理。当程序执行到循环控制语句时，根据循环判定条件对一组语句重复执行多次。循环结构可以看成是一个条件判断语句和一个向回转向语句的组合。另外，循环结构的三个要素：循环变量、循环体和循环终止条件。循环结构在程序框图中是利用判断框来表示，判断框内写上条件，两个出口分别对应着条件成立和条件不成立时所执行的不同指令，其中一个要指向循环体，然后再从循环体回到判断框的入口处。

3.2.1 LOOP语句

★基本LOOP语句中的CONTINUE语句

```
CREATE PROCEDURE p_3_2_2 AS
  x NUMBER := 0;
BEGIN
  LOOP -- After CONTINUE statement, control resumes here
    DBMS_OUTPUT.PUT_LINE ('Inside loop: x = ' || TO_CHAR(x));
    x := x + 1;
    IF x < 3 THEN
      CONTINUE;
    END IF;
    DBMS_OUTPUT.PUT_LINE('Inside loop, after CONTINUE: x = ' || TO_CHAR(x));
    EXIT WHEN x = 5;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE (' After loop: x = ' || TO_CHAR(x));
END;
/
--Result:
```

```
--Inside loop: x = 0
--Inside loop: x = 1
--Inside loop: x = 2
--Inside loop, after CONTINUE: x = 3
--Inside loop: x = 3
--Inside loop, after CONTINUE: x = 4
--Inside loop: x = 4
--Inside loop, after CONTINUE: x = 5
--After loop: x = 5
```

★基本LOOP语句中的CONTINUE WHEN语句

```
CREATE PROCEDURE p_3_2_3 AS
  x NUMBER := 0;
BEGIN
  LOOP -- After CONTINUE statement, control resumes here
    DBMS_OUTPUT.PUT_LINE ('Inside loop: x = ' || TO_CHAR(x));
    x := x + 1;
    CONTINUE WHEN x < 3;
    DBMS_OUTPUT.PUT_LINE('Inside loop, after CONTINUE: x = ' || TO_CHAR(x));
    EXIT WHEN x = 5;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE (' After loop: x = ' || TO_CHAR(x));
END;
/
--Result:
--Inside loop: x = 0
--Inside loop: x = 1
--Inside loop: x = 2
--Inside loop, after CONTINUE: x = 3
--Inside loop: x = 3
--Inside loop, after CONTINUE: x = 4
--Inside loop: x = 4
--Inside loop, after CONTINUE: x = 5
--After loop: x = 5
```

3.2.2 FOR LOOP语句

★基本LOOP语句

```
CREATE PROCEDURE p_3_2_4 AS
BEGIN
  DBMS_OUTPUT.PUT_LINE ('lower_bound < upper_bound');
  FOR i IN 1..3 LOOP
    DBMS_OUTPUT.PUT_LINE (i);
  END LOOP;

```



```
DBMS_OUTPUT.PUT_LINE ('lower_bound = upper_bound');
FOR i IN 2..2 LOOP
    DBMS_OUTPUT.PUT_LINE (i);
END LOOP;
DBMS_OUTPUT.PUT_LINE ('lower_bound > upper_bound');
FOR i IN 3..1 LOOP
    DBMS_OUTPUT.PUT_LINE (i);
END LOOP;
END;
/
--Result:
--lower_bound < upper_bound
--1
--2
--3
--lower_bound = upper_bound
--2
--lower_bound > upper_bound
--3
--2
--1
```

★反向FOR LOOP语句

```
CREATE PROCEDURE p_3_2_5 AS
BEGIN
    DBMS_OUTPUT.PUT_LINE ('upper_bound > lower_bound');
    FOR i IN REVERSE 1..3 LOOP
        DBMS_OUTPUT.PUT_LINE (i);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE ('upper_bound = lower_bound');
    FOR i IN REVERSE 2..2 LOOP
        DBMS_OUTPUT.PUT_LINE (i);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE ('upper_bound < lower_bound');
    FOR i IN REVERSE 3..1 LOOP
        DBMS_OUTPUT.PUT_LINE (i);
    END LOOP;
END;
/
--Result:
--upper_bound > lower_bound
--3
--2
--1
```

```
--upper_bound = lower_bound
--2
--upper_bound < lower_bound
--1
--2
--3
```

★FOR LOOP语句中模拟STEP子句

```
CREATE PROCEDURE p_3_2_6 AS
    step PLS_INTEGER := 5;
BEGIN
    FOR i IN 1..3 LOOP
        DBMS_OUTPUT.PUT_LINE (i*step);
    END LOOP;
END;
/
--Result:
--5
--10
--15
```

★FOR LOOP语句尝试更改索引值

```
CREATE PROCEDURE p_3_2_7 AS
BEGIN
    FOR i IN 1..3 LOOP
        IF i < 3 THEN
            DBMS_OUTPUT.PUT_LINE (TO_CHAR(i));
        ELSE
            i := 2;
        END IF;
    END LOOP;
END;
/
--Result:
-- 660: Loop variable(i) cannot be modified.
--Error in line 7
--Near character position 6
```

★FOR LOOP语句索引的外部语句引用

```
CREATE PROCEDURE p_3_2_8 AS
BEGIN
    FOR i IN 1..3 LOOP
        DBMS_OUTPUT.PUT_LINE ('Inside loop, i is ' || TO_CHAR(i));
    END LOOP;
    DBMS_OUTPUT.PUT_LINE ('Outside loop, i is ' || TO_CHAR(i));
END;
```

```
END;  
/  
--Result:  
-- 667: Variable(i) not declared.  
--Error in line 6  
--Near character position 57
```

★FOR LOOP语句索引与变量名称相同

```
CREATE PROCEDURE p_3_2_9 AS  
  i NUMBER := 5;  
BEGIN  
  FOR i IN 1..3 LOOP  
    DBMS_OUTPUT.PUT_LINE ('Inside loop, i is ' || TO_CHAR(i));  
  END LOOP;  
  DBMS_OUTPUT.PUT_LINE ('Outside loop, i is ' || TO_CHAR(i));  
END;  
/  
--Result:  
--Inside loop, i is 1  
--Inside loop, i is 2  
--Inside loop, i is 3  
--Outside loop, i is 5
```

★FOR LOOP语句引用与索引相同名称的变量

```
CREATE PROCEDURE p_3_2_10 AS  
BEGIN  
  <<main>> -- Label block.  
  DECLARE  
    i NUMBER := 5;  
  BEGIN  
    FOR i IN 1..3 LOOP  
      DBMS_OUTPUT.PUT_LINE ('local: ' || TO_CHAR(i) || ', global: ' || TO_CHAR(main.i) );  
      -- Qualify reference with block label.  
    END LOOP;  
  END main;  
END;  
/  
--Result:  
--local: 1, global: 5  
--local: 2, global: 5  
--local: 3, global: 5
```

★具有相同的索引名称的嵌套FOR LOOP语句

```
CREATE PROCEDURE p_3_2_11 AS  
BEGIN
```

```
<<outer_loop>>
FOR i IN 1..3 LOOP
  <<inner_loop>>
  FOR i IN 1..3 LOOP
    IF outer_loop.i = 2 THEN
      DBMS_OUTPUT.PUT_LINE('outer: ' || TO_CHAR(outer_loop.i) || ' inner: ' ||
TO_CHAR(inner_loop.i));
    END IF;
  END LOOP inner_loop;
END LOOP outer_loop;
END;
/
--Result:
--outer: 2 inner: 1
--outer: 2 inner: 2
--outer: 2 inner: 3
```

★FOR LOOP语句边界

```
CREATE PROCEDURE p_3_2_12 AS
  first INTEGER := 1;
  last INTEGER := 10;
  high INTEGER := 100;
  low INTEGER := 12;
BEGIN
  -- Bounds are numeric literals:
  FOR j IN -5..5 LOOP
    NULL;
  END LOOP;
  -- Bounds are numeric variables:
  FOR k IN REVERSE first..last LOOP
    NULL;
  END LOOP;
  -- Lower bound is numeric literal,
  -- Upper bound is numeric expression:
  FOR step IN 0..(TRUNC(high/low) * 2) LOOP
    NULL;
  END LOOP;
END;
/
```

★在运行时指定FOR LOOP语句边界

```
DROP TABLE temp;
CREATE TABLE temp (
  emp_no NUMBER, email_addr VARCHAR2(50)
```

```
);

CREATE PROCEDURE p_3_2_13 AS
    emp_count NUMBER;
BEGIN
    SELECT COUNT(employee_id) INTO emp_count
    FROM employees;
    FOR i IN 1..emp_count LOOP
        INSERT INTO temp (emp_no, email_addr) VALUES(i, 'to be added later');
    END LOOP;
END;
/
```

★在FOR LOOP语句中的EXIT WHEN语句

```
CREATE PROCEDURE p_3_2_14 AS
    v_employees employees%ROWTYPE;
    CURSOR c1 is SELECT * FROM employees;
BEGIN
    OPEN c1;
    -- Fetch entire row into v_employees record:
    FOR i IN 1..10 LOOP
        FETCH c1 INTO v_employees;
        EXIT WHEN c1%NOTFOUND;
        -- Process data here
    END LOOP;
    CLOSE c1;
END;
/
```

★在内部FOR LOOP语句中的EXIT WHEN语句

```
CREATE PROCEDURE p_3_2_15 AS
    v_employees employees%ROWTYPE;
    CURSOR c1 is SELECT * FROM employees;
BEGIN
    OPEN c1;
    -- Fetch entire row into v_employees record:
    <<outer_loop>>
    FOR i IN 1..10 LOOP
        -- Process data here
        FOR j IN 1..10 LOOP
            FETCH c1 INTO v_employees;
            EXIT outer_loop WHEN c1%NOTFOUND;
            -- Process data here
        END LOOP;
    END LOOP;
```

```
END LOOP outer_loop;
CLOSE c1;
END;
/
```

★在内部FOR LOOP语句中的CONTINUE WHEN语句

```
CREATE PROCEDURE p_3_2_16 AS
  v_employees employees%ROWTYPE;
  CURSOR c1 is SELECT * FROM employees;
BEGIN
  OPEN c1;
  -- Fetch entire row into v_employees record:
  <<outer_loop>>
  FOR i IN 1..10 LOOP
    -- Process data here
    FOR j IN 1..10 LOOP
      FETCH c1 INTO v_employees;
      CONTINUE outer_loop WHEN c1%NOTFOUND;
      -- Process data here
    END LOOP;
  END LOOP outer_loop;
  CLOSE c1;
END;
/
```

3.2.3 WHILE LOOP语句

```
CREATE PROCEDURE p_3_2_17 AS
  done BOOLEAN := FALSE;
BEGIN
  WHILE done LOOP
    DBMS_OUTPUT.PUT_LINE ('This line does not print.');
```

done := TRUE; -- This assignment is not made.

```
END LOOP;
WHILE NOT done LOOP
  DBMS_OUTPUT.PUT_LINE ('Hello, world!');
```

done := TRUE;

```
END LOOP;
END;
/
--Result:
--Hello, world!
```

3.3 顺序控制

3.3.1 NULL语句

★NULL 语句

程序体中的NULL 语句不会执行任何操作，并且会直接将控制传递到下一条语句。使用NULL 主要是为了提高PL/SQL的可读性。

```
CREATE PROCEDURE p_3_3_5 AS
  v_job_id VARCHAR2(10);
  v_emp_id NUMBER(6) := 110;
BEGIN
  SELECT job_id INTO v_job_id
  FROM employees
  WHERE employee_id = v_emp_id;
  IF v_job_id = 'SA_REP' THEN
    UPDATE employees
    SET commission_pct = commission_pct * 1.2;
  ELSE
    NULL; -- Employee is not a sales rep
  END IF;
END;
/
```

★在子程序创建期间，NULL语句作为占位符

```
CREATE OR REPLACE PROCEDURE award_bonus (
  emp_id NUMBER, bonus NUMBER) AUTHID DEFINER AS
BEGIN -- Executable part starts here
  NULL; -- Placeholder(raises "unreachable code" if warnings enabled)
END award_bonus;
/
```

★简单CASE语句的ELSE子句中的NULL语句

```
CREATE OR REPLACE PROCEDURE print_grade (grade CHAR) AUTHID DEFINER AS
BEGIN
  CASE grade
    WHEN 'A' THEN DBMS_OUTPUT.PUT_LINE('Excellent');
    WHEN 'B' THEN DBMS_OUTPUT.PUT_LINE('Very Good');
    WHEN 'C' THEN DBMS_OUTPUT.PUT_LINE('Good');
    WHEN 'D' THEN DBMS_OUTPUT.PUT_LINE('Fair');
    WHEN 'F' THEN DBMS_OUTPUT.PUT_LINE('Poor');
    ELSE NULL;
  END CASE;
END;
/

call print_grade('A');
call print_grade('S');
```

```
--Result:  
--Excellent
```

4 集合类型和RECORD类型

Gbase 8S 支持三种集合类型：关联数组、可变数组、嵌套表。

4.1 关联数据

关联数组是一组键值对。每个键都是一个唯一的索引，用于定位与索引相关联的值。

语法

variable_name (index) 。

参数说明

index: 是字符串类型（VARCHAR2、VARCHAR）。索引按排序顺序存储。

应用样例：

```
> set environment sqlmode 'oracle';  
  
Environment set.  
  
> set serveroutput on;  
  
set serveroutput succeed.  
  
DECLARE  
    TYPE population IS TABLE OF NUMBER -- 定义关联数组类型  
        INDEX BY VARCHAR2(64);          -- 用字符类型做索引  
    city_population population;          -- 声明关联数组变量  
    i VARCHAR2(64);  
BEGIN  
    -- 给数组添加key:values  
    city_population('Smallville') := 2000;  
    city_population('Midland') := 750000;  
    city_population('Megalopolis') := 1000000;  
    -- 改变key='Smallville'的值  
    city_population('Smallville') := 2001;
```



```
-- 输出关联数组结果

i := city_population.FIRST;
WHILE i IS NOT NULL LOOP
    DBMS_Output.PUT_LINE
        ('Population of ' || i || ' is ' || city_population(i));
    i := city_population.NEXT(i);
END LOOP;
END;
/

PL/SQL procedure successfully completed.

Population of Megalopolis is 1000000.0000000000000000
Population of Midland is 750000.0000000000000000
Population of Smallville is 2001.0000000000000000
```

4.2 可变数组

变量（变量大小数组）是一个数组，其元素数量可以从零（空）到声明的最大大小不等。

语法

variable_name (index)

参数说明

index: 索引最低边界值是1，上限是当前元素的数量。上限会随着您添加或删除元素而变化，但不能超过最大大小。当您从数据库中存储和检索变量时，其索引和元素顺序保持不变。

应用样例：

```
DECLARE
    TYPE Foursome IS VARRAY(4) OF VARCHAR2(15); -- VARRAY type
    -- varray variable initialized with constructor:
    team Foursome := Foursome('John', 'Mary', 'Alberto', 'Juanita');
BEGIN
    team(3) := 'Pierre'; -- Change values of two elements
    DBMS_OUTPUT.PUT_LINE('--res 1--');
    FOR i IN 1..4 LOOP
        DBMS_OUTPUT.PUT_LINE(i || ' ' || team(i));
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('--res 1--');
```

```
team(4) := 'Yvonne';
DBMS_OUTPUT.PUT_LINE('--res 2--');
FOR i IN 1..4 LOOP
    DBMS_OUTPUT.PUT_LINE(i || ' ' || team(i));
END LOOP;
DBMS_OUTPUT.PUT_LINE('--res 2--');
team := Foursome('Arun', 'Amitha', 'Allan', 'Mae');
DBMS_OUTPUT.PUT_LINE('--res 3--');
FOR i IN 1..4 LOOP
    DBMS_OUTPUT.PUT_LINE(i || ' ' || team(i));
END LOOP;
DBMS_OUTPUT.PUT_LINE('--res 3--');
END;
/
```

PL/SQL procedure successfully completed.

--res 1--

1.John

2.Mary

3.Pierre

4.Juanita

--res 1--

--res 2--

1.John

2.Mary

3.Pierre

4.Yvonne

--res 2--

--res 3--

1.Arun

2.Amitha

3.Allan

4.Mae

--res 3--

4.3 嵌套表

嵌套表是一种列类型，它以不特定顺序存储未指定数量的行。

语法

variable_name (index)

参数说明

index: 从数据库中存储和检索嵌套表时, 嵌套表的索引和行顺序可能不会固定。

限制

不支持多行结果bulk collection into语句赋值, 如例1。

支持单行结果对嵌套表列的into variable用法赋值, 如例2。

支持单行结果对嵌套表列的into record用法赋值, 但需要单独指定字段, 不能用table%rowtype, 如例3。

不支持 the(subquery)语法。

应用样例

```
> create type tp1 is table of int;  
> /
```

Oracle Type created.

```
> create table t1(c1 int,c2 tp1) nested table c2 store as nest_c2;
```

Table created

插入1条数据。

```
insert into t1 values(1,tp1(1,2,3,4));
```

1 row(s) inserted.

例1:不支持多行结果bulk collection into语句赋值

```
declare  
type tp2 is table of tp1;  
ini_tp2 tp2;  
begin  
    select c2 bulk collect into ini_tp2 from t1;  
    dbms_output.put_line(ini_tp2(1)(1));  
end;
```

201: A syntax error has occurred.

例2:支持单行结果对嵌套表列的into variable用法赋值

```
declare  
    ini_tp2 tp1;  
begin  
    select c2 into ini_tp2 from t1 where rownum=1;  
    dbms_output.put_line(ini_tp2(1));  
end;
```

PL/SQL procedure successfully completed.

1

例3:支持单行结果对嵌套表列的into record用法赋值

```
declare
  type re is record(c1 int,c2 tp1);
  ini_tp2 re;
begin
  select * into ini_tp2 from t1 where rownum=1;
  dbms_output.put_line(ini_tp2.c2(1));
end;
PL/SQL procedure successfully completed.
```

1

5 静态SQL

静态SQL是指SQL语法直接写在兼容PL/SQL语句中一个的特性。即把SQL语句直接写到程序里。写到变量里时，是动态SQL（见第6章）。除了特别说明，静态SQL和SQL语法相同。

静态SQL一般用于嵌入式SQL应用，在程序运行前，SQL语句必须是确定的。一般直接写在程序里。

5.1、游标

由系统构造和管理的游标是隐式游标，由用户构造和管理的游标为显式游标。隐式游标和显式游标都可以从他的属性获得会话游标信息。在PL/SQL中，通常显式游标是在第一次打开时解析它，而隐式游标在第一次执行SQL语句中解析他。

游标表达式用游标操作符表示，返回的是一个嵌套在查询语句中的游标。使用游标表达式的时候需要显示关闭，否则会出现锁表。



➤ 游标表达式把子查询作为外层查询的一列；

游标表达式把查询转换成一个结果集，该结果集可以作为参数传递给流或者转换函数。

5.1.1 显式游标

显式游标由用户构造和管理。必须声明显示游标，并给他命名和关联一个查询语句。显示游标不支持赋值，不能在表达式中使用，不能作为子程序的参数或宿主变量使用。如不得不使用，则选择使用游标变量来实现。

★声明显式游标

```
CURSOR cursor_name [ RETURN return_type ] IS select_statement;
```

```
CREATE OR REPLACE PROCEDURE p_4_2_3_1 AS
  CURSOR c1 IS
    SELECT employee_id, job_id, salary FROM employees WHERE salary > 2000;
  CURSOR c2 RETURN departments%ROWTYPE IS
    SELECT * FROM departments WHERE department_id = 110;
  CURSOR c3 IS
    SELECT * FROM departments WHERE department_id = 110;
BEGIN
  NULL;
END;
/
```

★Open and Close

声明显式游标之后，可以用OPEN语句打开，这个过程系统会分配数据库资源给此查询。处理查询：识别结果集，如果查询引用变量或游标参数，它的值会受影响。如果查询有FOR UPDATE子句，会锁住结果集。将游标定位在结果集的第一行之前。限制：同一个游标不能多次打开或关闭。只能打开一次关闭一次。

```
create table t1(id int,col varchar(30));
create table t2(id int,col varchar(30));
insert into t1 values(1,'a');
insert into t1 values(2,'b');

create or replace procedure p p_4_2_3_2 IS
  v1 int;
  v2 varchar(30);
  cursor c1 is select * from t1;
begin
  open c1;
  loop
    fetch c1 into v1,v2;
    exit when c1%notfound;
    insert into t2 values(v1,v2);
  end loop;
  close c1;
end;
/
```

★Fetch

打开一个显式游标后，能够用FETCH获得结果集中的每条数据。

```
FETCH cursor_name INTO into_clause
```

其中INTO_clause可以是变量列表或者是单个记录变量。对于查询返回的每一列，变量列表或记录变量必须有对应的兼容类型的变量。%TYPE和%ROWTYPE对声明在FETCH语句中的变量或记录变量非常有用。

FETCH语句获得结果集的当前行，将当前行的值存到变量或记录变量中，然后将光标下移到下一行。

通常在循环中使用FETCH语句，该循环当FETCH语句运行了所有的行后退出，为了检测退出条件，可以使用%NOTFOUND属性。FETCH语句没有返回值时，PL/SQL不会引起异常。

```
CREATE OR REPLACE PROCEDURE p_p_4_2_3_1 AS
    v_lastname employees.last_name%TYPE; -- variable for last_name
    v_jobid employees.job_id%TYPE; -- variable for job_id
    v_employees employees%ROWTYPE; -- record variable for row of table

    CURSOR c1 IS
        SELECT last_name, job_id FROM employees WHERE job_id='AD_PRES' ORDER BY last_name;
    CURSOR c2 IS
        SELECT * FROM employees WHERE job_id='AD_PRES' ORDER BY job_id;
BEGIN
    OPEN c1;
    LOOP -- Fetches 2 columns into variables
        FETCH c1 INTO v_lastname, v_jobid;
        EXIT WHEN c1%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_lastname);
    END LOOP;
    CLOSE c1;
    DBMS_OUTPUT.PUT_LINE('-----');

    OPEN c2;
    LOOP -- Fetches entire row into the v_employees record
        FETCH c2 INTO v_employees;
        EXIT WHEN c2%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_employees.last_name);
    END LOOP;
    CLOSE c2;
END;

/
--Atkinson          ST_CLERK
--Bell               SH_CLERK
--Bissot             ST_CLERK
--PL/SQL procedure successfully completed.
```

```
CREATE OR REPLACE PROCEDURE p_4_2_3_2 AS
  CURSOR c IS
    SELECT e.job_id, j.job_title FROM employees e, jobs j
    WHERE e.job_id = j.job_id AND e.manager_id = 100 ORDER BY last_name;
  -- Record variables for rows of cursor result set:
  job1 c%ROWTYPE;
  job2 c%ROWTYPE;
  job3 c%ROWTYPE;
  job4 c%ROWTYPE;
  job5 c%ROWTYPE;
BEGIN
  OPEN c;
  FETCH c INTO job1; -- fetches first row
  FETCH c INTO job2; -- fetches second row
  FETCH c INTO job3; -- fetches third row
  FETCH c INTO job4; -- fetches fourth row
  FETCH c INTO job5; -- fetches fifth row
  CLOSE c;
  DBMS_OUTPUT.PUT_LINE(job1.job_title || ' (' || job1.job_id || ')');
  DBMS_OUTPUT.PUT_LINE(job2.job_title || ' (' || job2.job_id || ')');
  DBMS_OUTPUT.PUT_LINE(job3.job_title || ' (' || job3.job_id || ')');
  DBMS_OUTPUT.PUT_LINE(job4.job_title || ' (' || job4.job_id || ')');
  DBMS_OUTPUT.PUT_LINE(job5.job_title || ' (' || job5.job_id || ')');
END;
/
--Result:
--Sales Manager (SA_MAN)
--Administration Vice President (AD_VP)
--Sales Manager (SA_MAN)
--Stock Manager (ST_MAN)
--Marketing Manager (MK_MAN)
--PL/SQL procedure successfully completed.
```

★显式游标查询中的变量

与游标变量相关的查询可以引用相同作用域的任何变量。当用OPEN打开游标变量时，可以用查询中的任意变量来标识结果集，对于变量的后续修改，会修改结果集。

```
CREATE OR REPLACE PROCEDURE p_p_4_2_3_4 AS
  sal employees.salary%TYPE;
  sal_multiple employees.salary%TYPE;
  factor INTEGER := 2;
  CURSOR c1 IS SELECT salary, salary*factor FROM employees WHERE job_id LIKE 'AD_%';
BEGIN
  OPEN c1; -- PL/SQL evaluates factor
```

```
LOOP
    FETCH c1 INTO sal, sal_multiple;
    EXIT WHEN c1%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('factor = ' || factor);
    DBMS_OUTPUT.PUT_LINE('sal = ' || sal);
    DBMS_OUTPUT.PUT_LINE('sal_multiple = ' || sal_multiple);
    factor := factor + 1; -- Does not affect sal_multiple
END LOOP;
CLOSE c1;
END;
/
--Result:
--factor = 2
--sal = 4400
--sal_multiple = 8800
--factor = 3
--sal = 24000
--sal_multiple = 72000
--factor = 4
--sal = 17000
--sal_multiple = 68000
--factor = 5
--sal = 17000
--sal_multiple = 85000
```

★显式游标查询中的别名

查询操作需要游标FETCH到一个记录变量里，而记录变量必须使用%ROWTYPE，且投影列存在虚拟列（即表达式），则该列必须有别名。且别名必须加AS，否则报错。在ORACLE中可以加AS,也可以不加。

在程序中引用定义的虚拟列

```
CREATE OR REPLACE PROCEDURE p_p_4_2_3_5 AS
    CURSOR c1 IS SELECT employee_id,(salary*.05) as raise FROM employees
        WHERE job_id LIKE '%_PRES' ORDER BY employee_id;
    emp_rec c1%ROWTYPE;
BEGIN
    OPEN c1;
    LOOP
        FETCH c1 INTO emp_rec;
        EXIT WHEN c1%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE ('Raise for employee #' || emp_rec.employee_id || ' is $' || emp_rec.raise);
    END LOOP;
    CLOSE c1;
```



```

END;
/
--Result:
--Raise for employee #114 is $550
--Raise for employee #120 is $400
--Raise for employee #121 is $410
--Raise for employee #122 is $395
--Raise for employee #123 is $325
--Raise for employee #124 is $368.445
--Raise for employee #145 is $700
--Raise for employee #146 is $675
--Raise for employee #147 is $600
--Raise for employee #148 is $550
--Raise for employee #149 is $525
--Raise for employee #201 is $650

```

★显式游标属性

显式游标和游标变量具有相同的属性。

%ISOPEN: 如果打开了游标则返回TRUE，否则返回FALSE。该属性用于：

- 在打开游标前用于检测显式游标是否打开。
- 在关闭游标前，检测是否已经打开。

```

CREATE OR REPLACE PROCEDURE p_p_4_2_3_6_1 AS
    CURSOR c1 IS SELECT last_name, salary FROM employees WHERE ROWNUM < 11;
    the_name employees.last_name%TYPE;
    the_salary employees.salary%TYPE;
BEGIN
    IF NOT c1%ISOPEN THEN
        OPEN c1;
    END IF;
    FETCH c1 INTO the_name, the_salary;
    IF c1%ISOPEN THEN
        CLOSE c1;
    END IF;
END;
/

```

%FOUND: 用于判断是否有取回的行。返回值：

- NULL 在打开显式游标之后，FETCH数据之前。
- TRUE 最近一个FETCH获得了一条数据
- FALSE 其他

```

CREATE OR REPLACE PROCEDURE p_p_4_2_3_6_2 AS

```

```
CURSOR c1 IS SELECT last_name, salary FROM employees
  WHERE ROWNUM < 11 ORDER BY last_name;
my_ename employees.last_name%TYPE;
my_salary employees.salary%TYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO my_ename, my_salary;
    IF c1%FOUND THEN -- fetch succeeded
      DBMS_OUTPUT.PUT_LINE('Name = ' || my_ename || ', salary = ' || my_salary);
    ELSE -- fetch failed
      EXIT;
    END IF;
  END LOOP;
END;
/
--Result:
--Name = Austin, salary = 4800
--Name = De Haan, salary = 17000
--Name = Ernst, salary = 6000
--Name = Faviet, salary = 9000
--Name = Greenberg, salary = 12008
--Name = Hunold, salary = 9000
--Name = King, salary = 24000
--Name = Kochhar, salary = 17000
--Name = Lorentz, salary = 4200
--Name = Pataballa, salary = 4800
```

%NOTFOUND: 用于退出循环

- NULL在打开显式游标之后，FETCH数据之前。
- FLASE最近一个FETCH获得了一条数据
- TRUE其他

```
CREATE OR REPLACE PROCEDURE p_4_2_3_6_3 AS
  CURSOR c1 IS  SELECT last_name, salary FROM employees
    WHERE ROWNUM < 11 ORDER BY last_name;
  my_ename employees.last_name%TYPE;
  my_salary employees.salary%TYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO my_ename, my_salary;
    IF c1%NOTFOUND THEN -- fetch failed
```

```
        EXIT;
    ELSE -- fetch succeeded
        DBMS_OUTPUT.PUT_LINE('Name = ' || my_ename || ', salary = ' || my_salary);
    END IF;
END LOOP;
END;
/
--Result:
--Name = Austin, salary = 4800
--Name = De Haan, salary = 17000
--Name = Ernst, salary = 6000
--Name = Faviet, salary = 9000
--Name = Greenberg, salary = 12008
--Name = Hunold, salary = 9000
--Name = King, salary = 24000
--Name = Kochhar, salary = 17000
--Name = Lorentz, salary = 4200
--Name = Pataballa, salary = 4800
```

%ROWCOUNT: 返回值

- ZERO: 在打开显式游标之后，FETCH数据之前
- 其他: FETCH获得了行数

```
CREATE OR REPLACE PROCEDURE p_4_2_3_6_4 AS
    CURSOR c1 IS SELECT last_name FROM employees
        WHERE ROWNUM < 11 ORDER BY last_name;
    name employees.last_name%TYPE;
BEGIN
    OPEN c1;
    LOOP
        FETCH c1 INTO name;
        EXIT WHEN c1%NOTFOUND OR c1%NOTFOUND IS NULL;
        DBMS_OUTPUT.PUT_LINE( name);
        IF c1%ROWCOUNT = 5 THEN
            DBMS_OUTPUT.PUT_LINE('--- Fetched 5th row ---');
        END IF;
    END LOOP;
    CLOSE c1;
END;
/
```

5.1.2 隐式游标

隐式游标由PL/SQL构造和管理。PL/SQL会在每次运行SELECT和DML语句时打开隐式游标，隐式游标支持的属性如下：

- SQL%ISOPEN Attribute: 游标是否打开
- SQL%FOUND Attribute: 是否有任意行受影响
- SQL%NOTFOUND Attribute: 有没有行受影响
- SQL%ROWCOUNT Attribute: 有多少行受影响

★SQL%ISOPEN

始终返回FALSE，因为隐式游标在他的关联语句执行完后永远是关闭的。

★SQL%FOUND

返回值：

- NULL 没有SELECT或DML运行
- TRUE 如果SELECT或DML返回或影响了一行或多行
- FALSE 其他情况

```
DROP TABLE dept_temp;
CREATE TABLE dept_temp AS SELECT * FROM departments;

CREATE OR REPLACE PROCEDURE p_4_2_1_2 (dept_no NUMBER) AS
BEGIN
    DELETE FROM dept_temp
    WHERE department_id = dept_no;
    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE ('Delete succeeded for department number ' || dept_no);
    ELSE
        DBMS_OUTPUT.PUT_LINE ('No department number ' || dept_no);
    END IF;
END;
/

call p(270);
call p(400);

--Result:
--Delete succeeded for department number 270
--No department number 400
```

★SQL%NOTFOUND

与SQL%FOUND的返回值相反

- NULL 如果没有SELECT或DML运行
- FALSE 如果SELECT或DML返回或影响了一行或多行

- TRUE 其他

该属性对PL/SQL中的SELECT INTO语句不起作用，因为调用集合函数的SELECT INTO总会有返回值（有时会为0），所以SQL%NOTFOUND始终为FALSE。

```
DROP TABLE dept_temp;
CREATE TABLE dept_temp AS SELECT * FROM departments;

CREATE OR REPLACE PROCEDURE p_4_2_1_3 (dept_no NUMBER) AS
BEGIN
    DELETE FROM dept_temp
    WHERE department_id = dept_no;
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE ('Delete succeeded for department number ' || dept_no);
    ELSE
        DBMS_OUTPUT.PUT_LINE ('No department number ' || dept_no);
    END IF;
END;
/

call p(270);
call p(400);

--Result:
--Delete succeeded for department number 270
--No department number 400
```

★SQL%ROWCOUNT

返回值：

- NULL：如果没有SELECT或DML语句执行
- 其他值：SELECT或DML返回或影响的行数

没有BULK COLLECT子句的SELECT INTO语句返回多条记录时，SQL/ROWCOUNT的值为1，而不是真实的行数。

```
DROP TABLE employees_temp;
CREATE TABLE employees_temp AS SELECT * FROM employees;
CREATE OR REPLACE PROCEDURE p_4_2_1_4 (dept_no NUMBER) AS
    mgr_no NUMBER(6) := 122;
BEGIN
    DELETE FROM employees_temp WHERE manager_id = mgr_no;

END;
/

--Result:
```

```
--Number of employees deleted: 8
```

5.2 处理结果集

游标可以用来处理查询结果集，可以用显式游标和隐式游标。前者只需用较少的代码，后者更为灵活。

下面的语句使用隐式游标

- 隐式游标FOR LOOP
- SELECT INTO

下面的语句使用显式游标

- 显式游标FOR LOOP
- OPEN、FETCH、CLOSE

5.2.1 SELECT INTO语句处理结果集

使用隐式游标，SELECT INTO 从一个或者多个数据库表获取数据（类似SQL的SELECT）然后把它们存储在变量中（SQL的SELECT做不到）。

处理单行结果集：

如果希望查询只返回一行，可以使用SELECT INTO语句存储该行的值到一个或多个变量中或到一个记录变量中；如果返回可能是多行，而你只关心第n行，则可以使用where ROWNUM 来限制结果集。

```
DROP TABLE IF EXISTS t;
DROP TABLE IF EXISTS tb;
CREATE TABLE t(id INT, name VARCHAR(20));
CREATE TABLE tb(id INT, name VARCHAR(20));
INSERT INTO t VALUES(1, 'a');
INSERT INTO t VALUES(2, 'b');

CREATE OR REPLACE PROCEDURE p_4_3_1_1 AS
    b1 INT;
BEGIN
    SELECT ID INTO b1 FROM T WHERE ID=2;
    INSERT INTO TB VALUES(b1,'S');
end;
/
```

5.2.2 FOR LOOP语句处理结果集

游标FOR LOOP允许你执行一条SELECT语句,然后循环结果集的所有行。游标FOR LOOP即可应用于显式游标也可以用于隐式游标,游标变量除外。

如果只在游标FOR LOOP循环中使用SELECT语句,则可以在游标FOR LOOP内部指定SELECT语句。这种形式的游标FOR LOOP循环称为隐式游标FOR LOOP语句。如果在相同的PL/SQL块中多次引用SELECT语句,则需要为它定义显式游标并在游标FOR LOOP语句中指定该游标。这种形式的游标FOR LOOP循环称为显式游标FOR LOOP语句。

隐式游标FOR LOOP声明的隐式变量为返回类型为%ROWTYPE记录类型变量,该记录对循环来说是内部的,并只在循环执行期间存在。循环中的语句可以引用变量,同时只能通过别名来引用虚拟列。当声明完循环变量时,FOR LOOP打开游标,每次循环 FOR LOOP都从结果集中获取一行存储在记录变量中,当没有行被获取时,游标FOR LOOP关闭游标。当循环中的语句控制权转移到外部或者PL/SQL产生异常时,游标也会关闭。

```
CREATE OR REPLACE PROCEDURE p_4_3_2_1 AS
BEGIN
  FOR item IN (
    SELECT last_name, job_id FROM employees
      WHERE job_id LIKE '%CLERK%' AND manager_id > 120
      ORDER BY last_name)
  LOOP
    DBMS_OUTPUT.PUT_LINE('Name = ' || item.last_name || ', Job = ' || item.job_id);
  END LOOP;
END;
/
--Result:
--Name = Atkinson, Job = ST_CLERK
--Name = Bell, Job = SH_CLERK
--Name = Bissot, Job = ST_CLERK
--...
--Name = Walsh, Job = SH_CLERK
```

显式 FOR LOOP Statement

```
CREATE OR REPLACE PROCEDURE p_4_3_2_2AS
CURSOR c1 IS
  SELECT last_name, job_id FROM employees
    WHERE job_id LIKE '%CLERK%' AND manager_id > 120 ORDER BY last_name;
BEGIN
  FOR item IN c1
  LOOP
    DBMS_OUTPUT.PUT_LINE('Name = ' || item.last_name || ', Job = ' || item.job_id);
  END LOOP;
```

```
END;  
/  
--Result:  
--Name = Atkinson, Job = ST_CLERK  
--Name = Bell, Job = SH_CLERK  
--Name = Bissot, Job = ST_CLERK  
--...  
--Name = Walsh, Job = SH_CLERK
```

5.2.3 OPEN-FETCH-CLOSE

为了完全控制查询结果集，需要声明显式游标并使用OPEN-FETCH-CLOSE并管理他们。这种处理数据集的技术非常复杂，但更加灵活。

使用多个游标，并行处理多个结果集；

在单次循环中处理多条数据，跳行或者将处理拆分成多个循环；

在一个PL/SQL块中指定查询，但在其他块中获得行。

注意：在游标变量和显式游标中都不可以用FETCH BULK COLLECT INTO

5.2.4 处理带有子查询的结果集

如果通过循环来处理结果集，并且对每一行都执行了另一个查询，则可以通过移除循环中的第二个查询，并使之成为第一个查询的子查询。普通的子查询对每个表进行评估，相关子查询对每一行进行评估。

```
CREATE PROCEDURE p_4_3_4_1 AS  
  CURSOR c1 IS  
  SELECT t1.department_id, department_name, staff  
  FROM departments t1,  
       ( SELECT department_id, COUNT(*) AS staff  
         FROM employees  
         GROUP BY department_id  
       ) t2  
  WHERE (t1.department_id = t2.department_id) AND staff >= 5  
  ORDER BY staff;  
BEGIN  
  FOR dept IN c1  
  LOOP  
    DBMS_OUTPUT.PUT_LINE ('Department = ' || dept.department_name || ', staff = ' || dept.staff);  
  END LOOP;  
END;  
/
```



```
--Result:
--Department = IT, staff = 5
--Department = Finance, staff = 6
--Department = Purchasing, staff = 6
--Department = Sales, staff = 34
--Department = Shipping, staff = 45
```

相关子查询

```
CREATE PROCEDURE p_4_3_4_2 AS
  CURSOR c1 IS
    SELECT department_id, last_name, salary
    FROM employees t
    WHERE salary > ( SELECT AVG(salary)
                     FROM employees
                     WHERE t.department_id = department_id
                   )
    ORDER BY department_id, last_name;
BEGIN
  FOR person IN c1
  LOOP
    DBMS_OUTPUT.PUT_LINE('Making above-average salary = ' || person.last_name);
  END LOOP;
END;
/
--Result:
--Making above-average salary = Hartstein
--Making above-average salary = Raphaely
--Making above-average salary = Bell
--...
--Making above-average salary = Higgins
```

5.3 游标变量

一个游标变量非常像一个显式游标。但也有自己的特性

- 游标变量不限于一个查询
- 可以给游标变量赋值
- 可以在表达式中使用游标变量
- 可以作为子程序的参数，可以使用游标变量在子程序之间传递结果集。只限于 SYS_REFCURSOR
- 不能接收参数，查询能够包括变量

★创建游标变量

创建游标变量：

- 定义REF CURSOR类型，然后声明该类型的变量。
- 声明SYS_REFCURSOR的变量，一个游标变量被称为REF CURSOR(引用游标)

一个REF CURSOR类型基本的语句定义：

```
TYPE type_name IS REF CURSOR [ RETURN return_type ]
```

如果指定了return_type,则定义的引用游标变量和声明该类型的变量就是强类型，否则为弱类型。SYS_REFCURSOR类型和他的变量都为弱类型。对于强类型的游标变量，可以把返回指定类型的查询和他关联。对于弱类型的游标变量，可以与任何查询关联。弱类型的引用游标可以互相转换，也可以与SYS_REFCURSOR互相转换。可以将一个强类型的游标变量赋值给一个弱类型的游标变量；只有当2个强类型游标变量具有相同的类型，可以进行赋值。

在本次版本中 RETURN只是语法实现，所以游标类型实际都是弱游标。

```
CREATE OR REPLACE PROCEDURE p_4_4_1_1 AS
  TYPE empcurtyp IS REF CURSOR RETURN employees%ROWTYPE; -- strong type
  TYPE genericcurtyp IS REF CURSOR; -- weak type
  cursor1 empcurtyp; -- strong cursor variable
  cursor2 genericcurtyp; -- weak cursor variable
  my_cursor SYS_REFCURSOR; -- weak cursor variable
  TYPE deptcurtyp IS REF CURSOR RETURN departments%ROWTYPE; -- strong type
  dept_cv deptcurtyp; -- strong cursor variable
BEGIN
  NULL;
END;
/
```

```
CREATE OR REPLACE PROCEDURE p_4_4_1_2 AS
  TYPE EmpRecTyp IS RECORD (
    employee_id NUMBER,
    last_name VARCHAR2(25),
    salary NUMBER(8,2));
  TYPE EmpCurTyp IS REF CURSOR RETURN EmpRecTyp;
  emp_cv EmpCurTyp;
BEGIN
  NULL;
END;
/
```

★Open and Close

声明一个游标变量后，可以用OPEN FOR语句打开：

- 将游标变量和查询相互关联（通常查询返回多行），查询中可以包含绑定变量的占位符，它的值通过USING子句来指定。
- 处理查询：识别结果集。如果查询引用变量或游标参数，它的值会受影响。如果查询有FOR UPDATE子句，会锁住结果集。将游标定位在结果集的第一行之前。
- 使用其他的OPEN FOR再次打开游标变量之前，不需要关闭它。在重新打开游标变量后，与它关联的前一个查询会丢失。

注意：OPEN FOR 后跟的SQL语句必须加单引号

```
DROP TABLE IF EXISTS t1;
CREATE TABLE t1(id INT,col VARCHAR(10));
INSERT INTO t1 VALUES(123,'abc');
INSERT INTO t1 VALUES(456,'def');
DROP TABLE IF EXISTS t2;
CREATE TABLE t2(col VARCHAR(10));
DROP PROCEDURE IF EXISTS p;

CREATE OR REPLACE PROCEDURE p_4_4_2 AS
    p_id INT:=456;
    v_col VARCHAR(10);
    TYPE t1CurTyp IS REF CURSOR;
    cursor_a t1CurTyp;
BEGIN
    OPEN cursor_a for 'SELECT col FROM t1 WHERE id=?' USING p_id;
    LOOP
        FETCH cursor_a INTO v_col;
        EXIT WHEN cursor_a%NOTFOUND;
        INSERT INTO t2 VALUES(v_col);
    END LOOP;
    CLOSE cursor_a;
END;
/
CALL p();
SELECT * FROM t2;
```

★Fetch

打开一个显式游标后，能够用FETCH获得结果集中的每条数据。游标变量的返回类型必须和FETCH语句的INTO子句相兼容。

注意：在游标变量和显式游标中都不可以用FETCH BULK COLLECT INTO

```
CREATE OR REPLACE PROCEDURE p_4_4_3 AS
```

```

cv SYS_REFCURSOR; -- cursor variable
v_lastname employees.last_name%TYPE; -- variable for last_name
v_jobid employees.job_id%TYPE; -- variable for job_id
v_employees employees%ROWTYPE; -- record variable row of table
BEGIN
  OPEN cv FOR 'SELECT last_name, job_id FROM employees WHERE   job_id =      "AD_PRES" ORDER
BY last_name';
  LOOP -- Fetches 2 columns into variables
    FETCH cv INTO v_lastname, v_jobid;
    EXIT WHEN cv%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(   v_jobid );
  END LOOP;
  DBMS_OUTPUT.PUT_LINE( '-----' );
  OPEN cv FOR 'SELECT * FROM employees WHERE   employee_id = 121 ORDER BY last_name';
  LOOP -- Fetches entire row into the v_employees record
    FETCH cv INTO v_employees;
    EXIT WHEN cv%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_employees.job_id);
  END LOOP;
  CLOSE cv;
END;
/
--Result:
--Atkinson ST_CLERK
--Bell SH_CLERK
--Bissot ST_CLERK
--...
--Walsh SH_CLERK
-----
--Higgins AC_MGR
--Greenberg FI_MGR
--Hartstein MK_MAN
--...
--Zlotkey SA_MAN

```

★游标变量赋值

可以使用另外的游标变量或者宿主变量给游标变量赋值。

```
target_cursor_variable := source_cursor_variable;
```

如果source游标变量是打开的，赋值后target游标变量也是打开的。两个游标变量指向同一区域。

```
CREATE OR REPLACE PROCEDURE p_4_4_4_1 AS
  cv1 SYS_REFCURSOR;
```

```

cv2 SYS_REFCURSOR;
v_lastname employees.last_name%TYPE; -- variable for last_name
v_jobid employees.job_id%TYPE; -- variable for job_id
v_employees employees%ROWTYPE; -- record variable row of table
BEGIN
  OPEN cv1 FOR 'SELECT last_name, job_id FROM employees WHERE  job_id =    "AD_PRES" ORDER
BY last_name';
  cv2 := cv1 ;
  LOOP
    FETCH cv2 INTO v_lastname, v_jobid;
    EXIT WHEN cv2%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(  v_jobid );
  END LOOP;
  DBMS_OUTPUT.PUT_LINE( '-----' );
  CLOSE cv2;
END;
/

```

下面示例中，先把cv1赋值给cv2。再关闭游标变量cv1，当再次打开后，在oracle中，游标变量cv2从现象上来看依然与cv1保持一致（指向新的SQL语句）；本次版本在这种场景下，cv2会处于不确定状态，可能指向旧的游标上下文，也可能指向cv1新打开的游标上下文。要想再次引用cv2，只能给cv2重新赋值。

```

CREATE OR REPLACE PROCEDURE p_4_4_4_2 AS
  cv1 SYS_REFCURSOR;
  cv2 SYS_REFCURSOR;
  v_lastname employees.last_name%TYPE; -- variable for last_name
  v_jobid employees.job_id%TYPE; -- variable for job_id
  v_employees employees%ROWTYPE; -- record variable row of table
BEGIN
  OPEN cv1 FOR 'SELECT last_name, job_id FROM employees WHERE  job_id =    "AD_PRES" ORDER
BY last_name';
  cv2 := cv1 ;
  close cv1;
  OPEN cv1 FOR 'SELECT last_name, job_id FROM employees WHERE  job_id =    "AD_PRES" ORDER
BY last_name';
  LOOP
    FETCH cv2 INTO v_lastname, v_jobid;
    EXIT WHEN cv2%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(  v_jobid );
  END LOOP;
  DBMS_OUTPUT.PUT_LINE( '-----' );
  CLOSE cv1;

```

```
END;  
/
```

★游标查询中的变量

与游标变量相关的查询可以引用他的作用域中的任意变量。当使用OPEN FOR打开游标变量时，PL/SQL会计算查询中的任意变量来标识结果集，对于变量的后续修改，不会改变结果集。若要更改结果集，必须更改变量的值，然后再次为同一查询打开游标变量。

```
CREATE OR REPLACE PROCEDURE p_4_4_5_1 AS  
    sal employees.salary%TYPE;  
    sal_multiple employees.salary%TYPE;  
    factor INTEGER := 2;  
    cv SYS_REFCURSOR;  
BEGIN  
    OPEN cv FOR 'SELECT salary, salary*? FROM employees WHERE job_id LIKE "AD_PRES"' using factor ;  
    LOOP  
        FETCH cv INTO sal, sal_multiple;  
        EXIT WHEN cv%NOTFOUND;  
        DBMS_OUTPUT.PUT_LINE('factor = ' || factor);  
        DBMS_OUTPUT.PUT_LINE('sal = ' || sal);  
        DBMS_OUTPUT.PUT_LINE('sal_multiple = ' || sal_multiple);  
        factor := factor + 1; -- Does not affect sal_multiple  
    END LOOP;  
    CLOSE cv;  
END;  
/  
--Result:  
--factor = 2  
--sal = 4400  
--sal_multiple = 8800  
--factor = 3  
--sal = 24000  
--sal_multiple = 48000  
--factor = 4  
--sal = 17000  
--sal_multiple = 34000  
--factor = 5  
--sal = 17000  
--sal_multiple = 34000
```

```
CREATE OR REPLACE PROCEDURE p_4_4_5_2 AS  
    sal employees.salary%TYPE;  
    sal_multiple employees.salary%TYPE;
```

```
factor INTEGER := 2;
cv SYS_REFCURSOR;
BEGIN
  DBMS_OUTPUT.PUT_LINE('factor = ' || factor);
  OPEN cv FOR 'SELECT salary, salary*? FROM employees WHERE job_id LIKE "AD_PRES"' using factor;
  LOOP
    FETCH cv INTO sal, sal_multiple;
    EXIT WHEN cv%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('sal = ' || sal);
    DBMS_OUTPUT.PUT_LINE('sal_multiple = ' || sal_multiple);
  END LOOP;

  factor := factor + 1;

  DBMS_OUTPUT.PUT_LINE('factor = ' || factor);

  OPEN cv FOR 'SELECT salary, salary*? FROM employees WHERE job_id LIKE "AD_PRES"' using factor;
  LOOP
    FETCH cv INTO sal, sal_multiple;
    EXIT WHEN cv%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('sal = ' || sal);
    DBMS_OUTPUT.PUT_LINE('sal_multiple = ' || sal_multiple);
  END LOOP;
  CLOSE cv;
END;
/
--Result:
--factor = 2
--sal = 4400
--sal_multiple = 8800
--sal = 24000
--sal_multiple = 48000
--sal = 17000
--sal_multiple = 34000
--sal = 17000
--sal_multiple = 34000
--factor = 3
--sal = 4400
--sal_multiple = 13200
--sal = 24000
--sal_multiple = 72000
--sal = 17000
--sal_multiple = 51000
```

```
--sal = 17000
--sal_multiple = 51000
```

★游标变量的属性

游标变量和显式游标变量具有相同的属性。

★子函数的参数

游标变量可以作为子程序参数用于在子程序间传递结果集：可以在一个子程序中打开游标变量，在其他子程序中处理他。当声明一个游标变量作为子函数的参数时：

- 如果子程序打开或者给游标变量赋值，则参数需要为IN OUT类型；
- 如果子程序只是获取数据，关闭游标变量，则参数可以为IN 或 IN OUT类型；

注意：只有SYS_REFCURSOR游标变量支持。REF CURSOR不支持。

```
DROP TABLE IF EXISTS T2;
CREATE TABLE T2(ID INT,COL VARCHAR(10));
INSERT INTO T2 VALUES(111,'AAA');
INSERT INTO T2 VALUES(222,'BBB');
DROP TABLE IF EXISTS T3;
CREATE TABLE T3(ID INT,COL VARCHAR(10));

CREATE OR REPLACE PROCEDURE p_4_4_7_1 (CURSOR_A OUT SYS_REFCURSOR) IS
BEGIN
    OPEN CURSOR_A FOR 'SELECT * FROM t2';
END;
/

CREATE OR REPLACE PROCEDURE p_4_4_7_2 IS
    v1 INT;
    v2 VARCHAR(10);
    cursor_b SYS_REFCURSOR;
BEGIN
    pro2(cursor_b);
    LOOP
        FETCH cursor_b INTO v1,v2;
        EXIT WHEN cursor_b%NOTFOUND;
        INSERT INTO t3 VALUES(v1,v2);
    END LOOP;
END;
/
```

5.4 使用游标更新/删除数据

FOR UPDATE游标

在PLSQL内,当FOR UPDATE子句与游标类用法关联时,统称为FOR UPDATE 游标。FOR UPDATE 子句的相关使用详见《GBase 8s V8.8 SQL指南:语法.docx》。

可定义FOR UPDATE游标的游标类型包括隐式游标、显式游标、游标变量及FOR LOOP 游标。

带有FOR UPDATE 子句的SELECT 游标,打开游标时,结果集中的行被锁定。当提交或回滚,游标关闭,被锁定的行解锁;由于隐式游标在语句运行完毕后保持关闭,所以FOR UPDATE应用于隐式游标总是被解锁的。缺省的锁等待为nowait。

语句定义：

```
--隐式游标
<select_statement> FOR UPDATE [OF column [,column]]

--显式游标
CURSOR <cursor_name> [ RETURN return_type ] IS <select_statement> FOR UPDATE [OF column
[,column]]

--游标变量
OPEN <cursor_var_name> FOR <select_statement> FOR UPDATE [OF column [,column]]

--FOR LOOP 游标
FOR <record_var> IN (<select_statement> FOR UPDATE [OF column [,column]]
) LOOP
```

扩展子句WHERE CURRENT OF

可以使用游标更新或删除结果集中的数据。若需要使用游标更新或删除数据,则在游标关联的查询语句中一定要使用FOR UPDATE 选项。

当游标拨动到需要更新或删除的行时,就可以使用 UPDATE/DELETE 语句进行数据更新/删除。此时必须在 UPDATE/DELETE 语句结尾使用WHERE CURRENT OF 子句,以限定删除/更新游标当前所指的行。

语句定义：

```
--更新
<update_statement> WHERE CURRENT OF <cursor_name>

--删除
<delete_statement> WHERE CURRENT OF <cursor_name>
```

示例

使用游标更新表中数据

```
declare
v_name varchar(20);
cursor v_cursor is select name from employee for update;
begin
    open v_cursor;
    loop
        fetch v_cursor into v_name ;
        exit when v_cursor%notfound;
        update employee set job='job' where current of v_cursor;
    end loop;
    close v_cursor;
commit;
end;
/
```

使用游标删除表中数据

```
declare
v_name varchar(20);
cursor v_cursor is select name from employee where job='teacher' for update;
begin
    open v_cursor;
    loop
        fetch v_cursor into v_name ;
        exit when v_cursor%notfound;
        delete from employee where current of v_cursor;
    end loop;
    close v_cursor;
commit;
end;
/
```

6 动态SQL

PL/SQL执行SQL语句时，有的SQL语句只能在运行阶段才能建立，例如当查询条件为用户输入时，SQL引擎无法在编译期对该语句进行确定，只有在用户输入一定的查询条件才能提交给SQL引擎进行处理，这种方式被称作动态SQL语句。

动态SQL是在运行时产生和执行SQL语句的一种编程方法。下列情况下动态SQL是非常有用的。

- 像编写ad hoc查询系统一样，编写一个多用途和灵活性非常强的应用程序。

- 编写必须用DDL语句的应用程序。
- 在编译时并不知道完整的SQL语句。
- 在编译时并不知道输入输出变量的类型和数量。

可以用EXECUTE IMMEDIATE语句处理大多数的动态SQL语句，EXECUTE IMMEDIATE语句最多返回一行结果集。

如果动态SQL语句返回结果集为多行，使用OPEN FOR, FETCH和CLOSE语句。

6.1 使用EXECUTE IMMEDIATE

EXECUTE IMMEDIATE支持DDL、DCL、DML以及单行SELECT语句。

SELECT语句如果返回单行，结果集放在 INTO 子句中；如果返回多行，结果集放在 BULK COLLECT INTO 子句中，当结果集返回多行时需要用集合类型（嵌套表，可变数组）接收。

如果动态SQL语句包含绑定变量占位符，则每个占位符必须在EXECUTE IMMEDIATE语句的USING子句有对应的绑定变量。

用USING 定义输入变量，USING子句不能使用NULL作为变量。为了解决此限制，在需要使用NULL的地方使用无初始化的变量来替代。USING子句不支持select语句。不要有SQL引擎不支持类型的绑定变量。

语法：

EXECUTE IMMEDIATE	---动态SQL的起始标示，表示是动态SQL调用
SQL 子句	---动态SQL语句
	方式1：提前声明一个SQL语句
	方式2：可以通过”添加SQL语句
USING 子句	---用于定义输入变量

例子

- 提前声明SQL的例子

CREATE OR REPLACE PROCEDURE p_5_1_1 AS
sql1 VARCHAR (100);
BEGIN
sql1 := 'UPDATE emp SET col=? where id=?';
EXECUTE IMMEDIATE sql1 USING 'zhangsan-q',100;
END;

```
/
```

- 通过”添加SQL语句例子

```
CREATE OR REPLACE PROCEDURE p_5_1_2 AS
    sql1 VARCHAR (100);
BEGIN
    EXECUTE IMMEDIATE 'UPDATE emp SET col=? where id=? ' USING 'zhangsan-q',100;
END;
/
```

- 使用USING子句的例子

```
DROP TABLE IF EXISTS emp2;
CREATE TABLE emp2(col1 VARCHAR(20),col2 INT);

CREATE or REPLACE PROCEDURE p_5_1_3(v_col1 varchar(20),v_col2 INT) AS
BEGIN
    EXECUTE IMMEDIATE 'INSERT INTO emp2 VALUES(?,?)' USING v_col1, v_col2;
END;
/
CALL p_5_1_3 ('2232',100);
```

- 使用动态SQL批量赋值,361版本该功能尚不完善。

```
drop table t1 cascade constraints;
create table t1(id int,c1 varchar2(20),c2 number(10,2));
insert into t1 values(1,'test1',1.1),(1,'test2',2.2);

create or replace procedure p1(v_id int)
is
type deftype1 is table of int;
tp1 deftype1;
v_sql varchar2(200);
begin
v_sql:='select id from t1 where id=:v_id';
execute immediate v_sql bulk collect into tp1 using v_id ;
for i in tp1.first .. tp1.last
loop
dbms_output.put_line(tp1(i));
end loop;
end;
/
```

过程已创建。

```
SQL> call p1(1);
```

```
1
```

```
1
```

调用完成。

6.2 使用游标语法

如果一个动态SQL语句是一个返回多行的SELECT语句，需要使用游标处理，提供了如下处理方法：

- 定义游标：使用sys_refcursor定义游标变量
- 打开游标：OPEN FOR语句把一个游标变量和动态SQL语句关联
- 使用游标：使用FETCH语句获取结果集
- 关闭游标：使用CLOSE语句关闭游标变量

例子：

```
CREATE OR REPLACE PROCEDURE p_5_2 IS
  cursor_a SYS_REFCURSOR;
  v_id INT;
  test_using INT:=2;
BEGIN
  OPEN cursor_a FOR 'SELECT * FROM t1 WHERE id = ?' USING test_using;
  LOOP
    FETCH cursor_a INTO v_id;
    EXIT WHEN cursor_a%NOTFOUND;
    INSERT INTO t2 VALUES(v_id);
  END LOOP;
  CLOSE cursor_a;
END;
/
```

7 子程序

PL/SQL子程序是可以重复调用的PL/SQL命名块。如果子程序有参数，每次调用的参数值可以不同。

子程序一般指过程和函数。通常，使用过程执行操作，使用函数计算并返回值。

7.1 使用子程序的原因

子程序支持开发和维护可靠的、可重用的代码。具有以下特点：

- 模块化：子程序允许将程序分解为可管理的、定义良好的模块。
- 更简单的应用程序设计：在设计应用程序时，可以推迟子程序的实现细节，直到测试了主程序，然后一步一步地细化它们。
- 易维护：您可以更改子程序的实现细节，而无需更改其调用程序。
- 用包进行管理：子程序可以分组到包中，关于这部分请参考包（PACKAGE）。
- 可复用：在许多不同的环境中应用程序可以使用相同的包子程序或独立子程序。
- 更优的性能：每个子程序都以可执行的形式编译和存储，可以重复调用。因为存储的子程序是在数据库服务器上运行的，所以通过网络进行一次调用就可以启动一个大型作业。这种分工减少了网络流量，缩短了响应时间。存储的子程序被缓存并在用户之间共享，这降低了内存需求和调用开销。

7.2 子程序调用

子程序调用的形式如下：

```
call subprogram_name [ ( [ parameter [, parameter]... ] ) ]
```

如果子程序没有参数，或者为每个参数指定了默认值，则可以省略参数列表或指定空参数列表。

7.3 子程序属性

每个子程序属性只能在子程序声明中出现一次。属性可以以任何顺序出现。属性出现在子程序标题中的IS或AS关键字之前。

7.4 子程序部分

子程序以子程序标题开始，该标题指定其名称和（可选）参数列表。子程序有以下几个部分：

- 声明部分（可选）：本部分声明并定义局部类型、游标、常量、变量、异常。
- 可执行部分（必需）：此部分包含一个或多个用于赋值、控制执行和操作数据的语句。
- 异常处理部分（可选）：此部分包含处理运行时错误的代码。

★函数的附加部分

函数的结构与过程相同，并且：

- 函数标题必须包含RETURN子句，该子句指定函数返回的值的数据类型（过程标题不能有RETURN子句）
- 在函数的可执行部分，每个执行路径都必须指向一个RETURN语句

例如：

```
CREATE FUNCTION p_6_4_1 (original NUMBER)
RETURN NUMBER
AS
    original_squared NUMBER;
BEGIN
    original_squared := original * original;
    RETURN original_squared;
END;
/
```

调用代码及结果如下：

```
SELECT p_6_4_1 (100) FROM dual;
```

结果:

```
10000.0000000000
```

★RETURN语句

RETURN语句立即结束包含它的子程序的执行。子程序可以包含多个返回语句。

- 函数中的RETURN语句：在函数中，每个执行路径必须指向一个RETURN语句，每个RETURN语句必须指定一个表达式。RETURN语句将表达式的值分配给函数标识符，并将控制权返回给调用程序，调用后立即恢复执行。

例如：

```
CREATE FUNCTION f_6_4_2 (n INTEGER)
RETURN INTEGER
IS
BEGIN
    IF n = 0 THEN
        RETURN 1;
    ELSIF n = 1 THEN
        RETURN n;
    ELSE
        RETURN n*n;
    END IF;
END;
```

```
END IF;  
END;  
/
```

调用代码及结果如下：

```
SELECT f_6_4_2 (0), f_6_4_2 (1), f_6_4_2 (2) FROM dual;
```

结果：

```
(expression) (expression) (expression)  
            1            1            4  
1 row(s) retrieved.
```

- 过程中的RETURN语句：在过程中，RETURN语句将控制权返回给调用程序，调用后立即恢复执行。过程中的RETURN语句不能指定表达式。

7.5 子程序参数

★形参和实参

如果希望子程序具有参数，请在子程序标题中声明形参。在每个形参声明中，指定参数的名称和数据类型，以及（可选）其模式和默认值。在子程序的执行部分，通过名称引用形参。

例如，过程raise_salary包含2个形参emp_id和amount：

```
CREATE PROCEDURE p_6_5_1 (  
    emp_id NUMBER,  
    amount NUMBER  
) IS  
BEGIN  
    UPDATE employees SET salary = salary + amount WHERE employee_id = emp_id;  
END p_6_5_1;
```

在调用子程序时，指定要将其值赋给形参的实参。相应的实参和形参必须具有兼容的数据类型。

例如，emp_num、bonus、merit为实参：

```
.....  
emp_num NUMBER(6) := 120;  
bonus NUMBER(6) := 100;  
merit NUMBER(4) := 50;  
BEGIN  
.....
```



```
p_6_5_1(emp_num, bonus);  
/* 执行的是如下语句:  
  
UPDATE employees SET salary = salary + 100 WHERE employee_id = 120; */  
p_6_5_1(emp_num, merit + bonus);  
/* 执行的是如下语句:  
  
UPDATE employees SET salary = salary + 150 WHERE employee_id = 120; */  
  
.....  
END;
```

★子程序参数传递方法

有两种将实参传递给子程序的方法：

- 传址：传递一个指向实参的指针，实参和形参引用相同的内存位置。
- 传值：将实参的值赋给相应的形参，实参和形参指的是不同的内存位置。

★子程序参数模式

形参的模式决定了它的行为。

- IN：缺省模式，将值传递给子程序
- OUT：必须指定，向调用程序返回一个值
- IN OUT：必须指定，将初始值传递给子程序，并将更新后的值返回给调用程序

不管OUT或IN-OUT参数是如何传递的：

- 如果子程序成功退出，那么实参的值就是赋给形参的最终值
- 如果子程序以异常结束，则实参的值未定义

7.6 子程序调用解析

子程序调用时，它首先在当前作用域中搜索匹配的子程序声明，然后根据需要在连续的封闭作用域中搜索。

如果它们的子程序名和参数列表匹配，则声明和调用匹配。如果声明中每个必需的形参在调用中都有相应的实参，则参数列表匹配。

如果编译器没有找到与调用匹配的声明，那么它将生成一个语义错误。

7.7 递归子程序

递归子程序调用自身。递归是一种强大的简化算法的技术。

递归子程序至少要有两条执行路径，一条指向递归调用，另一条指向终止条件。如果没有后者，递归将继续，直到PL/SQL耗尽内存并引发预定义的异常存储错误。

子程序的每次递归调用都会创建子程序声明的每个项及其执行的每个SQL语句的实例。

例如，下面函数实现阶乘：

```
CREATE FUNCTION f_6_7 (n INTEGER) RETURN INTEGER
IS
BEGIN
  IF n = 1 THEN
    RETURN n;
  ELSE
    RETURN n * factorial(n-1);
  END IF;
END;
```

调用代码及结果如下：

```
CREATE OR REPLACE PROCEDURE p_6_7 AS
BEGIN
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE(i || '!' || f_6_7(i));
  END LOOP;
END ;
/
call p_6_7();
```

Result:

```
1!=1
2!=2
3!=6
4!=24
5!=120
```

7.8 子程序副作用

如果子程序改变了它自己的局部变量的值以外的任何东西，它就会产生副作用。例如，更改以下任何一项的子程序都有副作用：

- 自身的输出或输入输出参数
- 全局变量
- 包中的公共变量

- 数据库表
- 数据库
- 外部状态（例如，通过调用DBMS输出）

副作用可能会阻止查询的并行化，产生依赖于顺序的（因此是不确定的）结果，或者要求跨用户会话维护包状态。

7.9 SQL语句可以调用的PL/SQL函数

为了能在SQL中被调用，存储的函数（及其调用的子程序）必须遵守以下规则，这些规则旨在控制函数副作用：

- 当从SELECT语句或是并发的INSERT、UPDATE、DELETE语句中调用函数时，它不能修改任何数据表。
- 当从INSERT、UPDATE、DELETE或MERGE语句中调用函数时，它不能查询或修改这些语句所能影响到的数据表。
- 当从SELECT，INSERT，UPDATE，DELETE或MERGE语句中调用时，子程序无法执行以下任何SQL语句：事务控制语句（例如COMMIT）、系统控制语句（例如ALTER SYSTEM）、自动提交的数据库定义语言（DDL）语句（例如CREATE）。

如果函数执行部分中的任何SQL语句违反了规则，则在解析该语句时会出现运行错误。

函数的副作用越少，则可以在SELECT语句中对其进行更好得优化。

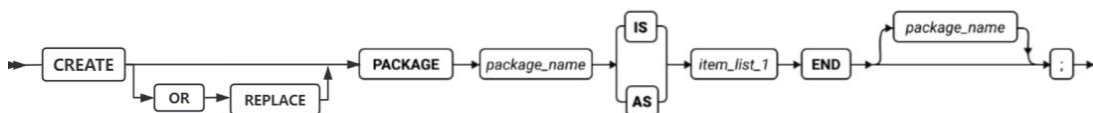
8 包

包是一组相关过程、函数、变量、常量、类型、游标和异常等PL/SQL程序设计元素的组合。包具有面向对象设计的特点，是对这些PL/SQL程序设计元素的封装。包像一个容器或一个命名空间，可以将各种逻辑相关的过程、函数、变量、常量、类型、游标和异常结合在一起。为开发人员编写大型复杂应用程序提供良好的组织单元。当包被定义好后，应用程序可以通过包来访问各种不同的功能单元，而不用担心过多零散的子程序导致程序代码的松散。包具有简化应用设计、提高应用性能、实现信息隐藏、子程序重载等特点。对包中涉及的过程、函数、变量、常量、类型、游标和异常等PL/SQL程序设计元素如无特殊约束说明，其用法和现状保持一致。

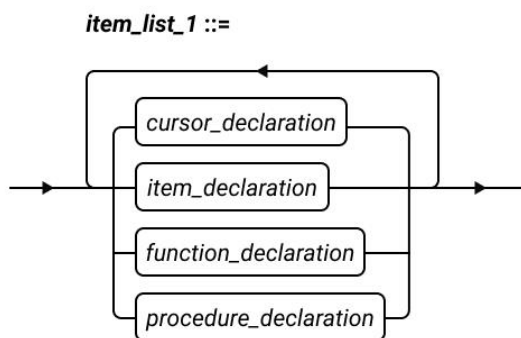
8.1 包的创建

一个PL/SQL包由如下两部分组成：包头，或称为包的规范：主要是包的一些定义信息，不包含具体的代码实现部分，也可以说包头是PL/SQL程序和其他应用程序的接口，包含子程序、变量、常量、类型、游标和异常的声明。包体：是对包规范中声明的子程序和游标的实现部分，包体的内容对于外部应用程序来说是不可见的，包体就像是一个黑匣子一样，是对包规范的实现。注意：包头中子程序和游标是需要实现部分的，如果包头中没有声明子程序或游标，则包体就不是必须的。

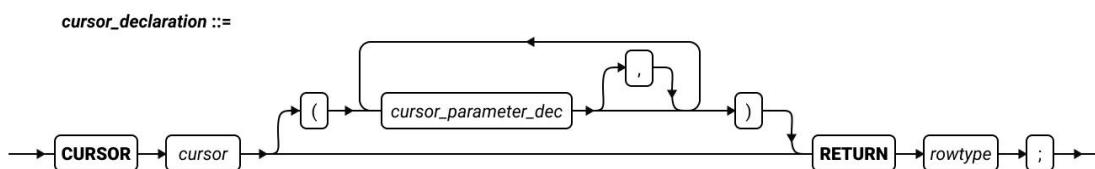
创建包头语法图如下：



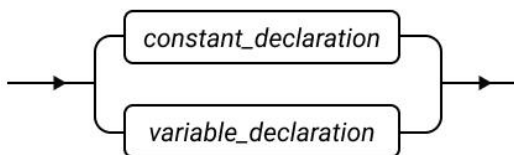
END后的package_name可以省略。



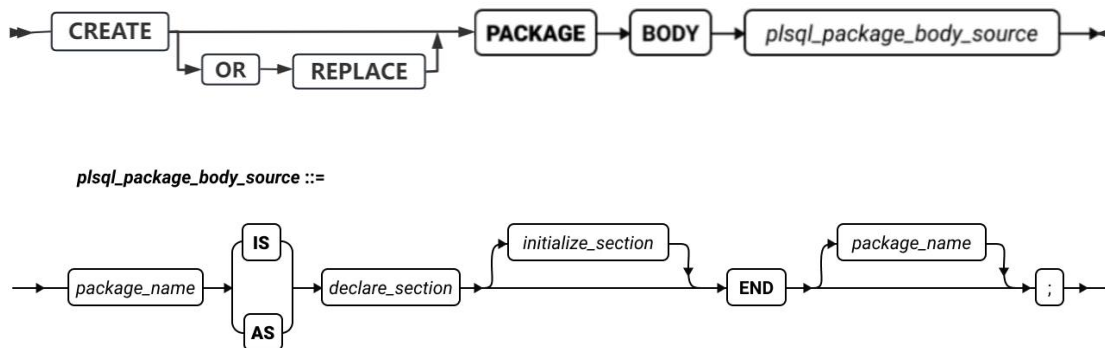
Item_list_1如上图，可以包含多种元素，也可以一个元素都不包括，即包头可以为空，不过这样在实际使用中没有意义。



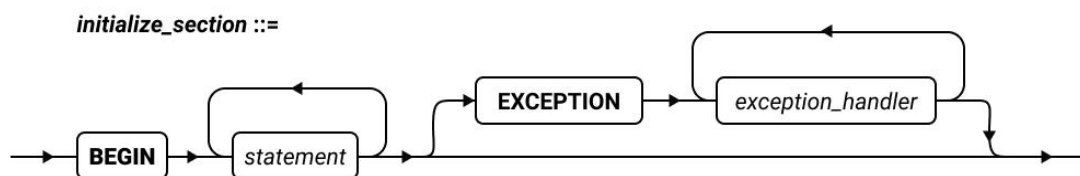
item_declaration ::=



创建包体语法如下：



`declare_section`定义和声明私有常量和变量，不允许外部引用，仅允许包内引用。`END`后的`package_name`可以省略。



包在第一次被引用时，数据库会为当前SESSION实例化这个包，然后会执行 `initialize_section` 部分。

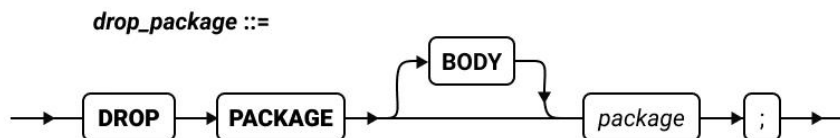
注意：包的子过程不使用包的变量时，不触发包的 `initialize_section` 部分。当引用包的过程、函数，且包体初始化部分执行失败时，已执行的包有状态成员（变量、常量、游标）的修改不会进行回滚，需要用户自己来维护数据一致性。

包头声明的变量、常量、游标等，包体只允许初始化一次。如重复初始化变量，返回报错 669: Variable(变量名) redeclared。

包头声明的游标名不允许与 `PROCEDURE`、`FUNCTION` 的例程名相同。如果重名，返回报错 6114: Previous use of '变量名' conflicts with this use。

8.2 包的删除

语法图如下：



`DROP PACKAGE`会将包头和包体一并删除，也可以通过 `DROP PACKAGE BODY`关键字只删除包体，此时包头依然有效。

例如：

```
DROP PACKAGE BODY pkg_Count;
```

8.3 包的引用

包在首次引用时实例化并执行initialize_section部分，包为SESSION级，即多SESSION之间是不同的包实例。

包的引用可以采用点标记的方式，例如：

```
CALL pkg_Count.getCount();
```

下面我们通过几个例子示范包的常见用法，注意需要将SQLMODE设置为'ORACLE'，如果需要输出请设置SERVEROUTPUT为ON。

```
SET ENVIRONMENT SQLMODE 'ORACLE';  
SET SERVEROUTPUT ON;
```

例子1：包中变量和常量的声明

```
CREATE PACKAGE pkg_7_3_1 AS  
    companyName CONSTANT VARCHAR(100) := 'GBASE';  
    productName VARCHAR(100);  
END;
```

本例子中，只有包头，并不需要包体。注意，常量不能使用变量赋初始值。然后我们创建一个过程，对包中声明的常量进行引用，包中的变量可以赋值和引用。

```
CREATE PROCEDURE p_7_3_1 AS  
BEGIN  
    DBMS_OUTPUT.PUT_LINE(pkg_7_3_1.companyName);  
    pkg_7_3_1.productName := 'GBase 8s';  
    DBMS_OUTPUT.PUT_LINE(pkg_7_3_1.productName);  
END;
```

通过如下代码进行过程的调用。

```
CALL PROCEDURE p_7_3_1 ();
```

可以看到如下结果。

```
GBASE  
GBase 8s
```

例子2：包中过程和函数的用法

```
CREATE PACKAGE pkg_7_3_2 AS  
    PROCEDURE helloWorld;  
    FUNCTION myPlus(param1 IN INT, param2 IN INT) RETURN INT;  
END;
```

然后我们创建包体，用于实现包头中声明的过程和函数。

```
CREATE PACKAGE BODY pkg_7_3_2 AS
  PROCEDURE helloWorld AS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('HelloWorld!');
  END;
  FUNCTION myPlus (param1 IN INT, param2 IN INT) RETURN INT AS
  BEGIN
    RETURN param1 + param2;
  EXCEPTION
    WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('error');
  END;
END;
```

我们创建一个过程，对包中声明的过程和函数进行引用。

```
CREATE PROCEDURE p_7_3_2 AS
BEGIN
  pkg_7_3_2.helloWorld;
  DBMS_OUTPUT.PUT_LINE(pkg_7_3_2.myPlus (2, 3));
END;
```

通过如下代码进行过程的调用。

```
CALL PROCEDURE p_7_3_2 ();
```

可以看到如下结果。

```
HelloWorld!
5
```

例子3：包的状态

```
CREATE PACKAGE pkg_7_3_3 AS
  PROCEDURE getCount;
END;

CREATE PACKAGE BODY pkg_7_3_3 AS
  FCount INTEGER;
  PROCEDURE getCount AS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Current count is:');
    DBMS_OUTPUT.PUT_LINE(FCount);
    FCount:= FCount + 1;
  END;
BEGIN
  FCount := 1;
END;
```

在上面的例子中，`pkg_7_3_3`在包头中声明了一个名称为`getCount`的过程，然后包体中声明了一个类型为`INTEGER`的私有变量，并对包头中声明的`getCount`过程进行了代码实现，`getCount`过程实现输出私有变量`FCount`的值，并每次对`Fcount`进行+1操作。由于包是`SESSION`级的，因此我们可以看到在`call pkg_7_3_3.getCount()`多次引用时返回值会发生变化：第一次引用时输出的是1，第二次引用时输出的是2，第三次引用时输出的是3。特别注意的是此时如果另一个`SESSION`中引用`pkg_7_3_3`，输出的不是4而是1，即多`SESSION`之间是不同的包实例。

例子4：通过包中游标对数据进行遍历

为了演示这个例子，我们需要创建一张包含2个字段的表。

```
CREATE TABLE TPERSON(  
    FID VARCHAR(10) NOT NULL PRIMARY KEY,  
    FName VARCHAR(50)  
)
```

并准备几条数据。

```
INSERT INTO TPERSON VALUES('1', 'first');  
INSERT INTO TPERSON VALUES('2', 'second');  
INSERT INTO TPERSON VALUES('3', 'third');
```

创建包头，包含一个返回类型是`TPERSON`表行记录类型的游标声明。

```
CREATE PACKAGE pkg_7_3_4 AS  
    CURSOR myc RETURN TPERSON%ROWTYPE;  
END;
```

创建包体，可以看到包体中对游标进行了代码实现。这里需要补充说明两点：一是包的实现部分也可以写在包头中，从而不创建包体，建议的方式是将游标的代码写在包体里，将定义和实现进行分离；二是对游标的`OPEN`、`FETCH`、`CLOSE`等操作一般建议写在调用包游标的地方。

```
CREATE PACKAGE BODY pkg_7_3_4 AS  
    CURSOR myc RETURN TPERSON%ROWTYPE IS SELECT FID, FName FROM TPERSON;  
END;
```

然后我们创建一个过程，在过程中，通过`TYPE`定义了一个记录类型`TR`，并声明了一个类型为`TR`的`mt`变量。在过程的实现部分，首先通过`OPEN`打开了游标，然后遍历取数据并赋值给`mt`，并通过内置包`DBMS_OUTPUT`的`PUT`方法和`PUT_LINE`方法进行输出。

```
CREATE PROCEDURE p_7_3_4 AS  
    TYPE TR IS RECORD(  
        id VARCHAR(10),  
        name VARCHAR(50)  
    );
```



```
mt TR;
BEGIN
  OPEN pkg_7_3_4.myc;
  FETCH pkg_7_3_4.myc INTO mt;
  WHILE pkg_7_3_4.myc%FOUND LOOP
    DBMS_OUTPUT.PUT(mt.id);
    DBMS_OUTPUT.PUT(' ');
    DBMS_OUTPUT.PUT_LINE(mt.name);
    FETCH pkg_7_3_4.myc INTO mt;
  END LOOP;
  CLOSE pkg_7_3_4.myc;
END;
```

最后我们通过如下代码进行过程的调用

```
CALL PROCEDURE p_7_3_4 ();
```

可以看到如下结果

```
1: first
2: second
3: third
```

9 DML触发器

兼容 ORACLE DML 触发器语法须 SQLMODE 在 ORACLE 兼容模式下。

```
set environment sqlmode 'oracle';
```

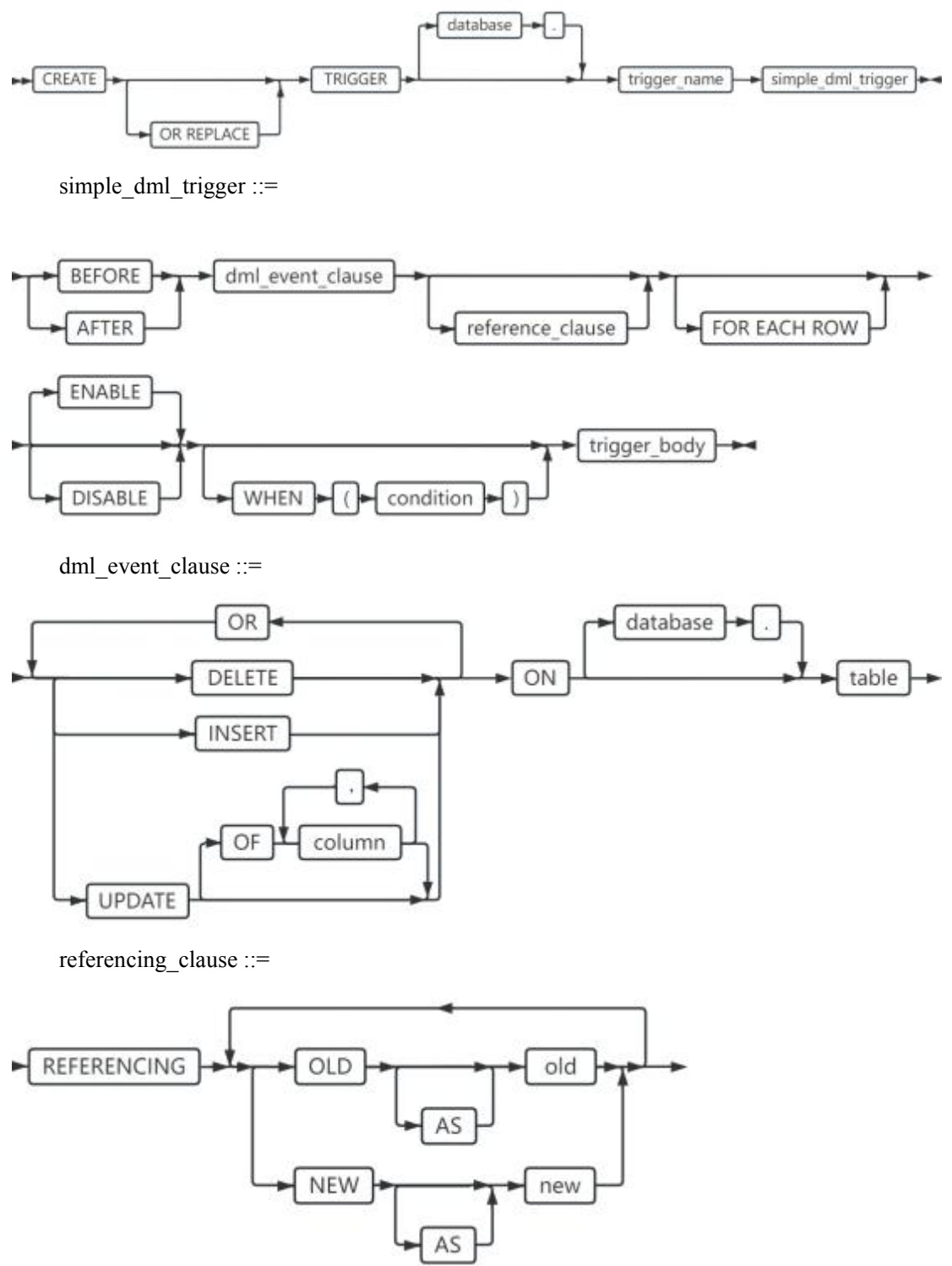
O 兼容 DML 触发器(TRIGGER)定义为当某些数据操作事件发生时，数据库应该采取的操作。在相关的事件发生时，由数据库自动地隐式地激发语句集合，即直到一个语句激发的所有触发器执行完成之后该语句才结束，而其中任何一个触发器执行的失败都将导致该语句的失败。O 兼容 DML 触发器与 O 兼容存储模块类似，都是在数据库上保存并执行的一段 **PLSQL 语句**。当触发器被触发时，数据库将执行保存的 PLSQL 语句。

当前版本以不同模式区分原生触发器与 Oracle 兼容触发器。O 兼容触发器在 gbase8s 模式下触发，数据库将返回报错。因此，为保证行为一致，涉及目标表的DML操作也需在相同模式下进行。例如，在 Oracle 模式下创建的触发器需保持在 Oracle 模式下对目标表进行INSERT、UPDATE 或 DELETE操作。如果在 gbase8s 模式下对目标表进行DML操作，则报错，但由于触发器的机制逻辑，原则上触发器报错不影响DML操作本身。

9.1 触发器的创建

使用 CREATE TRIGGER 语句在表上定义触发器。

创建触发器语法图如下：



元素	描述	限制
database	所在数据库	需要对应数据库的对应操作权限
trigger_name	新的触发器声明的名称	必须在当前数据库中的触发 器名

		称中是唯一的
condition	触发必须满足的逻辑条件	必须在行级触发器下使用，即必须与 FOR EACH ROW 一同使用
trigger_body	触发器被激活时执行的PLSQL语句块	必须在 Oracle 模式下
column	激活触发器的列	必须在触发 table 中存在
table	触发表的名称	必须在数据库中存在
old	在触发器操作中使用的旧引用声明的名称	在此 CREATE TRIGGER 语句中必须唯一
new	在触发器操作中使用的旧引用声明的名称	在此 CREATE TRIGGER 语句中必须唯一

触发器相关概念说明：

```
CREATE TRIGGER <触发器名称>
<触发时间> <触发事件> ON <触发对象> [<触发级别>]
<触发模式>
<触发条件>
<触发体> ;
```

- 触发事件：引起触发器被触发的 DML 语句事件，包括 INSERT 、 DELETE 、 UPDATE 。
- 触发时间：触发事件和该 TRIGGER 的操作顺序。 BEFORE 指明触发器在执行触发语句之前激发； AFTER 指明触发器在执行触发语句之后激发。
- 触发操作/触发体：即该 TRIGGER 被触发之后要执行的 PLSQL 语句块。
- 触发对象：被定义触发器的数据库对象，当前版本仅支持基表。
- 触发级别：即语句级触发器和行级触发器。
 - 语句级触发器：是指当某触发事件发生时，该触发器只执行一次；
 - 行级触发器：是指当某触发事件发生时，对受到该操作影响的每一行数据，触发器都单独执行一次，使用 FOR EACH ROW 定义。
- 触发条件：由 WHEN 子句指定一个逻辑表达式。必须在行级触发器下使用，即必须与 FOR EACH ROW 一同使用。只有当该表达式的值为 TRUE 时，遇到触发事件才会自动执行触发器，使其执行触发操作。
- 触发模式：设置触发器为启用(ENABLE)或禁用(DISABLE)。

存在如下限制：

- 指定 **OR REPLACE** 将创建一个新触发器或替换同名的现有触发器。
- 创建触发器的表必须存在于当前数据库中。
- 不支持在诊断表、违例表、外部数据库中的表、临时表、外部表、系统目录表创建触发器。

例如，表 **tab1** 表结构如下：

```
CREATE TABLE tab1 (C1 int,c2 varchar(20));  
CREATE TABLE tab2 (c1 int, c2 varchar(20));
```

插入如下数据：

```
INSERT INTO tab1 values(1,'test');  
INSERT INTO tab1 values(2,'test');
```

例子1, 在表 **tab1** 上创建 **BEFORE** 触发器, 插入 2 行数据至表 **tab1** , 触发器 **tri1** 将被激发 1 次：

```
CREATE OR REPLACE TRIGGER tri1 BEFORE INSERT ON tab1  
BEGIN  
    INSERT INTO tab2 values(null,'test');  
END;  
/  
  
----批插 2 条数据  
INSERT INTO tab1 SELECT * FROM tab1;  
  
----触发 1 次  
SELECT * FROM tab2;  
  
C1|C2 |  
--+----+  
|test|
```

例子2, 在表 **tab1** 上创建 **AFTER** 触发器, 插入 2 行数据至表 **tab1** , 触发器 **tri2** 将被激发一次：

```
CREATE OR REPLACE TRIGGER tri2 AFTER INSERT ON tab1  
BEGIN  
    INSERT INTO tab2 values(null,'test');  
END;  
/
```

```

-----批插 2 条数据

INSERT INTO tab1 SELECT * FROM tab1;

-----触发 1 次

SELECT * FROM tab2;

C1|C2 |
--+----+
|test|

```

例子3，在表 tab1 上创建 BEFORE 触发器，触发事件为 INSERT 、 UPDATE 或 DELETE ， 分别执行插入、更新、删除，触发器 tri3 被激发 3 次：

```

CREATE OR REPLACE TRIGGER tri3 BEFORE INSERT OR UPDATE OR DELETE ON tab1
BEGIN
    INSERT INTO tab2 values(null,'test');
END;
/
-----插入

INSERT INTO tab1 SELECT * FROM tab1;

-----更新

UPDATE tab1 SET c2='test1' WHERE c1 = 1;

-----删除

DELETE tab1 WHERE c1 = 1;

-----触发 3 次

SELECT * FROM tab2;

C1|C2 |
--+----+
|test|
|test|
|test|

```

例子4，在表 tab1 上创建 BEFORE FOR EACH ROW 行级触发器，插入 2 行数据至表 tab1 ， 触发器 tri4 将被激发 2 次：

```

CREATE OR REPLACE TRIGGER tri4 BEFORE INSERT ON tab1 FOR EACH ROW
BEGIN
    INSERT INTO tab2 values(null,'test')
END;

```

```

/
----批插 2 条数据
INSERT INTO tab1 SELECT * FROM tab1;

----触发 2 次
SELECT * FROM tab2;

C1|C2 |
---+---+
|test|
|test|

```

例子5，在表 tab1 上创建 BEFORE FOR EACH ROW 行级触发器，定义该触发器启用，并定义触发条件为 :NEW.c1>10，插入c1 值为10与11的 2 行数据，触发器 tri5 将被激发 1 次：

```

CREATE OR REPLACE TRIGGER tri5 BEFORE INSERT ON tab1 FOR EACH ROW ENABLE WHEN
(NEW.c1 > 10)
BEGIN
    INSERT INTO tab2 values(null,'test');
END;
/
----批插 2 条数据
INSERT INTO tab1 VALUES(10,'test');
INSERT INTO tab1 VALUES(11,'test');

----触发 1 次
SELECT * FROM tab2;

C1|C2 |
---+---+
|test|

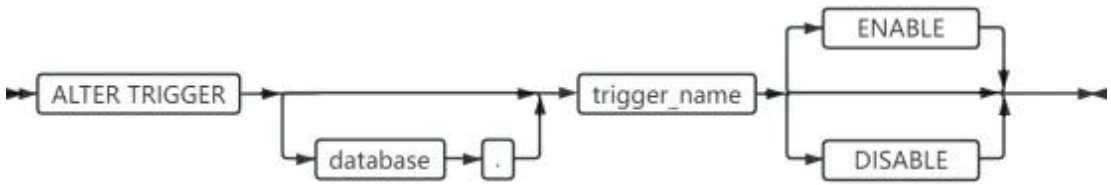
```

9.2 启用或禁用触发器

使用 ALTER TRIGGER 语句来启用或禁用表上的触发器。

- ENABLE 表示，如果发生触发事件，则数据库服务器执行触发操作。
- DISABLE 表示，触发事件不会导致执行触发操作，数据库服务器将忽略该触发器及其操作。

启用或禁用触发器语法图如下：



元素	描述	限制
database	所在数据库	需要对应数据库的对应操作权限
trigger_name	新的触发器声明的名称	必须在当前数据库中的触发器名称中是唯一的

例如，启用触发器 tri1：

```
ALTER TRIGGER tri1 ENABLE;
```

9.3 触发体

O 兼容 DML 触发器触发体定义基于 PLSQL 语法。

```
CREATE TRIGGER <触发器名称> <触发时间> <触发事件> ON <触发对象> [<触发级别>] <触发模式> <
触发条件> <触发体>;

<触发体> ::=

[ DECLARE <声明部分>]

BEGIN

<执行部分>

END [<触发器名称>];
```

当前版本已实现的 PLSQL 语法在触发体部分均支持。PLSQL特性详细边界描述及用例，详见本手册对应章节。

9.4 新、旧行值的引用

仅在行级触发器，可以使用新、旧行值的引用访问正在处理的行中的数据。缺省的引用名是 :OLD、:NEW，可使用 REFERENCING 子句修改。:OLD 表示记录被处理前的值，:NEW 表示记录被处理后的值。

在触发体内，使用以下语法引用新旧行的字段：

```
:引用名.列名
```

:OLD、:NEW 不同触发事件下含义如下：

触发事件	:OLD	:NEW
INSERT	报错处理	该语句结束时将插入的值
UPDATE	更新前的旧值	该语句结束时将更新的值
DELETE	删除前的旧值	报错处理

说明及限制：

1. 新、旧数据的引用，仅当触发器为行级触发器可使用，即使用 `FOR EACH ROW` 定义触发级别。当触发器为语句级时，单次触发可能涉及多行数据，系统无法定位新、旧数据的引用如何指向。
2. 省略 `REFERENCING` 子句，缺省引用为 `:OLD`、`:NEW`。
3. 引用不能出现在 `RECORD` 级的相关赋值操作中，需以 `:引用名.字段` 的形式使用。例如，`:NEW := NULL` 是不允许的。
4. 触发器不能更改`:OLD`的字段值。
5. `BEFORE` 触发器可以在触发事件`INSERT`将 `:NEW` 字段值放入表中之前更改它们。
`AFTER` 触发器不能更改 `:NEW`字段值，因为触发语句在触发器触发之前运行。
6. 如果同时定义了 `BEFORE INSERT` 触发器和 `AFTER INSERT` 触发器，并且 `BEFORE INSERT` 触发器更改了 `:NEW` 字段值，则 `AFTER INSERT` 触发器可见该更改。
7. `WHEN` 子句使用新旧引用，引用名依据 `REFERENCING` 子句定义。如无 `REFERENCING` 子句定义，缺省的引用名是 `:OLD`、`:NEW`。

触发器主要基于原生触发器进行改造兼容 Oracle DML 触发器。因此，在内部部分处理逻辑上，保持 GBase 8s 原生行为，**存在以下兼容差异：**

1. `INSERT` 的旧引用 `:OLD` 与`DELETE`的新引用 `:NEW` ，保持 GBase 8s 原生行为，创建触发器报错处理，返回报错信息。 Oracle 行为为置 `NULL` 处理。
2. GBase 8s 原生行级触发器，仅通过 `FOR EACH ROW` 定义，不支持以行为单位区分触发时间 (`BEFORE/AFTER`) 的功能。在`INSERT`、`UPDATE`、`DELETE` 几种触发事件下，原生 `FOR EACH ROW` 与 Oracle DML 触发器的对应关系如下：

GBase 8s 原生触发操作	等价对应Oracle
INSERT FOR EACH ROW	BEFORE INSERT FOR EACH ROW

不支持此功能	AFTER INSERT FOR EACH ROW
不支持此功能	BEFORE UPDATE FOR EACH ROW
UPDATE FOR EACH ROW	AFTER UPDATE FOR EACH ROW
不支持此功能	BEFORE DELETE FOR EACH ROW
DELETE FOR EACH ROW	AFTER DELETE FOR EACH ROW

本版本 **仅语法兼容** 不支持的功能分支。例如，当尝试创建 AFTER INSERT FOR EACH ROW 触发器时，系统表相关信息保持 AFTER INSERT FOR EACH ROW 。当相关 INSERT 操作激发该触发器时，实际触发时间为 BEFORE INSERT FOR EACH ROW。

3. 由于以上原因，本版本 O 兼容 DML 触发器需注意以下几点操作规范：
- a) 如果同一张表存在多个触发器，触发器创建顺序**必须保证 BEFORE FOR EACH ROW 先于 AFTER FOR EACH ROW 创建**。同类型触发器执行顺序，以创建先后为准。
 - b) 如果 BEFORE UPDATE FOR EACH ROW 触发器在触发事件 UPDATE 将 :NEW 字段值放入表中之前试图更改它，本版本执行合法，但 :NEW 引用值无法在表中生效，因为 UPDATE 操作已经完成。

10 自定义类型

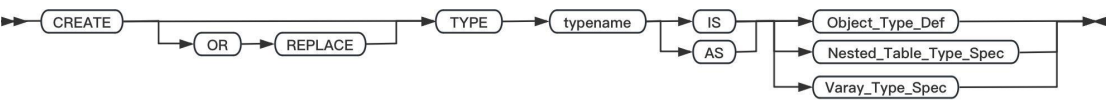
自定义类型包括对象类型、集合类型。嵌套表类型和可变数组统称为集合类型。

对象类型只能使用 CREATE TYPE 语句进行创建，然后在PLSQL中使用。集合类型可以使用CREATE TYPE 语句创建，也可以在PL\SQL中临时声明并使用。

可变数组是从0到声明的最大的尺寸的大小。访问时可以使用语法：
variable_name(index),index的最小值是1。访问到的值会因元素的删除或者增加而不同。

10.1 CREATE TYPE语句创建自定义类型

使用CREATE TYPE语句可以将自定义类型存储到数据库中复用。



参数说明：

元素	描述	限制	语法
typename	自定义类型名称	符合8s的命名规则，长度最长128字符	标识符

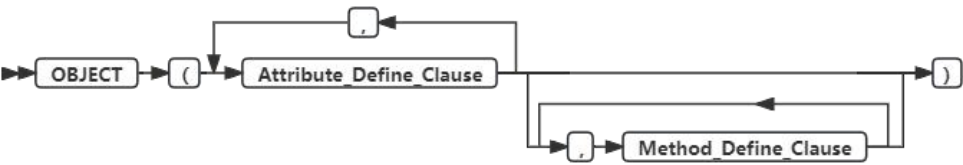
说明及限制：

- [OR REPLACE]如果存在同名的自定义类型则覆盖。8s仅语法支持。
- 8s本版本暂不支持使用ALTER TYPE修改已创建的对象类型。

10.1.1 对象类型

对象类型中包含方法及属性。

Object_Type_Def:=



说明及限制：

- 不支持创建对象表。
- 不支持定义为表中列。
- 不支持在SQL中使用对象类型。

Attribute_Define_Clause:=

属性定义在对象类型中是不可缺少的。



参数说明：

元素	描述	限制	语法
attribute_name	对象类型的属性名称	长度最长128字符，在创建的类型中必须是唯一的	标识符
datatype	属性对应的类型	包括8s中基础数据类型及支持的自定义类型	数据类型

例如：创建简单对象类型，只包含属性，不包含方法。

```
> create or replace type person_typ1 as object
```

```
(
    name varchar2(10),
    gender varchar2(10),
    birthdate date
);
/
Oracle Type created.
```

说明及限制：

- 不能在属性上增加NOT NULL约束。
- 不支持%rowtype和%type定义对象类型属性的数据类型。
- 不支持byte,text,serial,bigserial,serial8,boolean,timestamp with time zone定义对象类型属性的数据类型。
- 不支持集合类型list,set,multiset定义对象类型属性的数据类型。
- 不支持对象类型属性定义中指定ref关键字。
- 不支持对象类型属性是package中定义的集合类型。

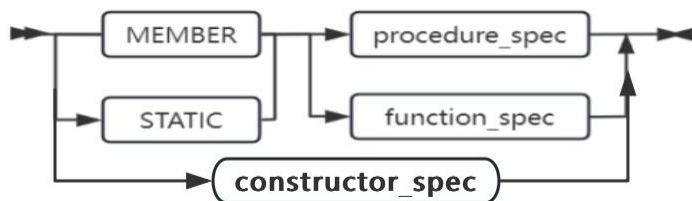
Method_Define_Clause

方法在对象类型中可以缺省，通常表示为过程和函数，在此处只进行方法的声明，具体实现需要在CREATE TYPE BODY中进行。

目前支持 MEMBER 方法和STATIC：

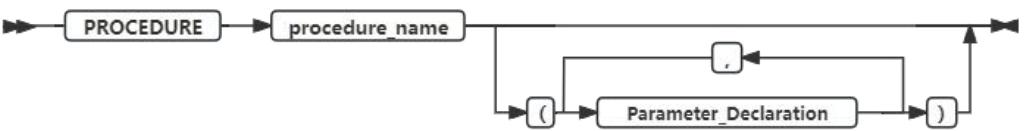
MEMBER 方法是需要对对象进行实例化才可以调用的方法，并且在 MEMBER 方法的方法体中有一个隐式的参数self，它表示调用该方法的对象，self参数可以隐式或显示引用。

STATIC方法可以在对象类型上执行全局操作，而不需要访问特定对象实例的数据。与MEMBER方法相比，STATIC方法只能由对象类型调用，不能由对象实例调用，且静态方法不会默认将self值作为第一个参数传入。



procedure_spec:=

以过程的形式声明对象类型的方法。

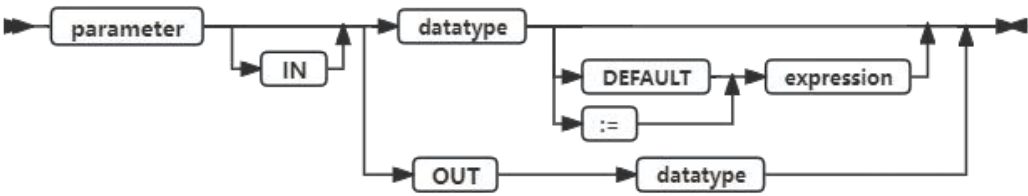


参数说明：

元素	描述	限制	语法
procedure_name	过程名称	必须满足名称限制的字符串	标识符

Parameter_Declaration:=

声明方法中的参数声明。

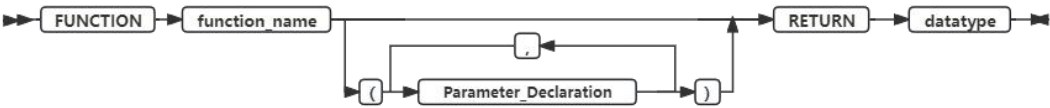


参数说明：

元素	描述	限制	语法
parameter	参数名称	必须满足名称限制的字符串	标识符
datatype	参数类型	支持8s中基础数据类型及支持的自定义类型	数据类型

function_spec:=

以函数的形式声明对象类型的方法



参数说明：

元素	描述	限制	语法
function_name	函数名称	必须满足名称限制的字符串	标识符

datatype	参数数据类型	类型在数据库中必须存在	数据类型
----------	--------	-------------	------

示例

分别创建一个包含了声明MEMBER过程和MEMBER函数的对象类型。

```
>create type tp1 as object(  
  c1 int,  
  c2 int,  
  member function f1(c1 decimal(18,2)) return decimal(18,2)  
);  
/  
Oracle Type created.  
  
>create type tp1 as object(  
  c1 int,  
  c2 int,  
  member procedure p1(c1 decimal(18,2))  
);  
/  
Oracle Type created.  
  
>create or replace type obj is object (  
  type_name char(20),  
  id int,  
  member function func2(self in out obj) return int  
);  
/  
Oracle Type created.
```

创建一个包含了声明STATIC方法的对象类型。

```
>create or replace type obj_typ is object (  
  id int,  
  static function get_id1(obj in out obj_typ) return int,  
  static procedure get_id2(obj in out obj_typ)  
);  
/  

```

创建一个同时包含MEMBER方法和STATIC方法的对象类型。

```
>create or replace type obj_typ is object (  

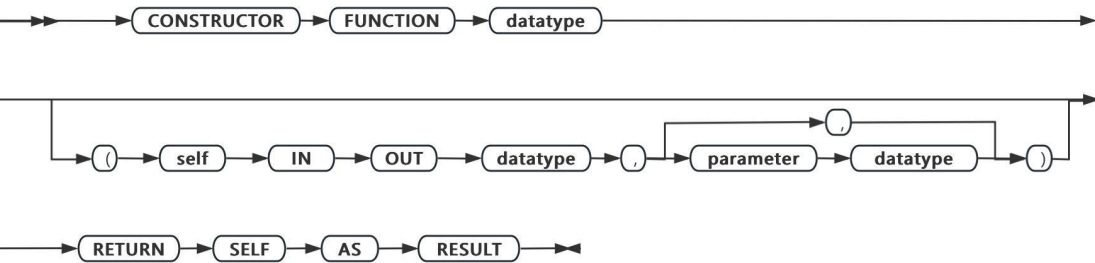
```

```
id int,  
static function get_id2(obj in out obj_typ) return int,  
member function get_id1(self in out obj_typ,p1 int) return int  
);  
/
```

说明及限制:

- 不支持MEMBER方法参数指定self in out nocopy。

constructor_spec::=
自定义构造函数用来个性化构造一个对象的实例。



参数说明:

元素	描述	限制	语法
function_name	函数名称	必须满足名称限制的字符串	标识符
datatype	参数数据类型	类型在数据库中必须存在	数据类型

用法及限制

- 自定义构造函数的名称必须与类型名称一致。
- 入参如果包含self参数，则self参数必须为in out，且self参数类型必须与创建的object类型相同。
- 返回值不以类型标识，使用特殊语法 return self as result。
- object 自定义构造函数必须为function。
- 创建类型头时允许声明重名函数。
- 不对类型头中定义的函数参数进行校验。
- 调用构造函数时，会默认在系统构造函数和所有自定义构造函数中寻找匹配度最高的

函数调用，匹配规则与member，static方法相同。

- 与系统默认相同，自定义构造函数后不允许存在其他表达式，例如constr().a 和constr()(a) 都会报错。

10.1.2 嵌套表类型

嵌套表类型是一组不同类型或者相同类型数据的集合，不限制集合元素个数。

Nested_Table_Type_Spec:=



例如：创建一个嵌套表类型，并在PLSQL中引用。

```
> create type mytype is table of varchar(20);
/

Oracle Type created.

declare
  c2 mytype;
begin
  c2:=mytype('SMTIH','ALIEN','SCOTT','TOM');
  dbms_output.put_line(c2(2));
end;
/

ALIEN

PL/SQL procedure successfully completed.
```

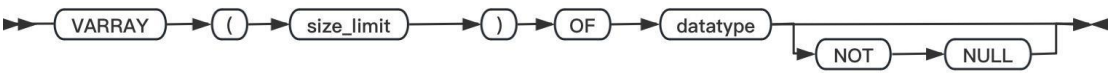
说明及限制:

- ORACLE模式下运行
- 类型不支持serial、serial8、bigserial数据类型
- 不支持定义为表中列。

10.1.3 可变数组类型

可变数组类型是一有序的类型相同的元素集合。

Varay_Type_Spec:=



参数说明：

元素	描述	限制	语法
size_limit	最大元素限制	整数值	数值
datatype	数据类型	基础数据类型以及自定义数据类型	标识符

例如：定义一个可变数组并在PLSQL中引用它。

```
create type mytype is varray(10) of varchar(20);
/

Oracle Type created.

declare
    c2 mytype;
begin
    c2:=mytype('SMTIH','ALIEN','SCOTT','TOM');
    dbms_output.put_line(c2(2));
end;
> /

PL/SQL procedure successfully completed.

ALIEN
```

说明及限制:

- ORACLE模式下运行
- 类型不支持serial、serial8、bigserial数据类型
- 不支持定义为表中列。

10.2 CREATE TYPE BODY语句创建自定义类型体

用来定义或实现使用CREATE TYPE语句创建的对象类型中定义的方法。

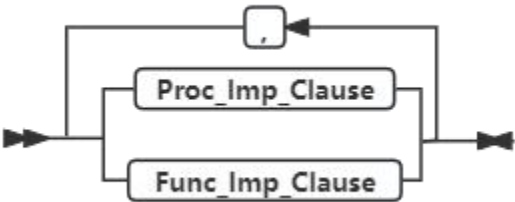


参数说明：

元素	描述	限制	语法
typename	对象体的名称	必须与创建的对象名称一致	标识符

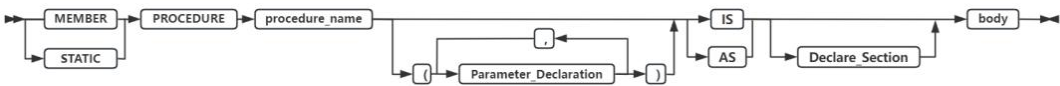
Subprog_Decl_In_Type:=

对象类型中定义的方法的类型。



10.2.1 存储过程实现

Proc_Imp_Clause:=



参数说明：

元素	描述	限制	语法
procedure_name	存储过程的名称	与对象类型声明中的存储过程名称一致	标识符

Declare_Section和body分别为声明部分和块的执行部分，详情请参考《GBase 8s V8.8 PLSQL 手册》中包的创建。

10.2.2 函数实现

Func_Imp_Clause:=

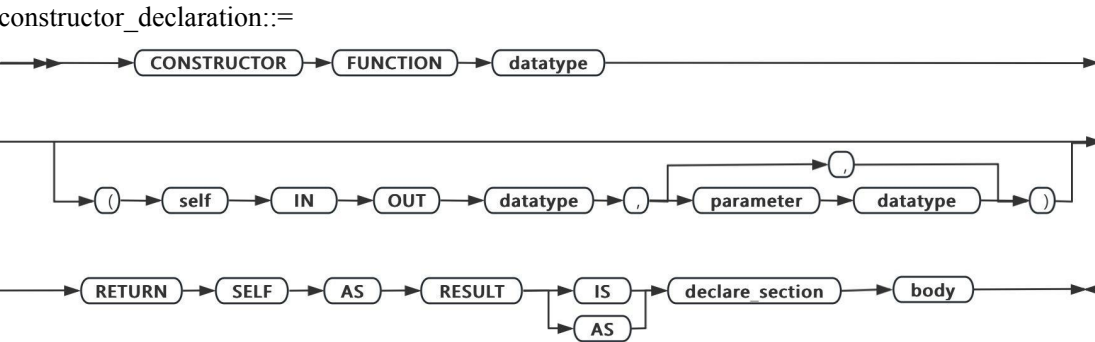
实现对象类型中声明的函数。


```
end;  
/
```

说明及限制:

- 不支持对象类型定义重名MEMBER方法或STATIC方法。
- 不支持定义方法名称为GBase特殊表达式，如volume，否则创建类型体会报错。

暂不支持使用DROP TYPE BODY语句删除已创建的类型体，删除对象类型自动删除对应的类型体。



参数说明:

元素	描述	限制	语法
datatype	函数的返回类型	包括8s中基础数据类型及支持的自定义类型	数据类型
declare_section	变量声明部分	与存储过程函数限制相同	表达式
body	程序主体	与存储过程函数限制相同	表达式

用法及限制

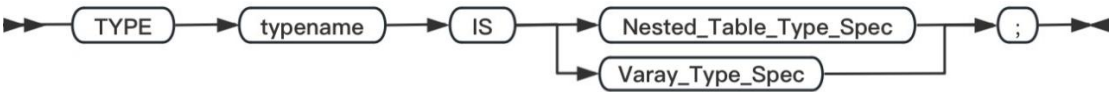
- 自定义构造函数的名称必须与类型名称一致。
- 入参如果包含self参数，则self参数必须为in out，且self参数类型必须与创建的object类型相同。
- 返回值不以类型标识，使用特殊语法 return self as result。
- 构造函数类型体中的return语句不允许接表达式，使用'return'即可。
- 创建类型体时不允许声明重名函数。
- 若出现类型头中函数的参数个数与类型体中的参数个数不一致时，函数参数以类型体为准。

应用样例

```
create or replace type color as object(  
    red int,  
    constructor function color(self in out color) return self as result,  
    member function to_string return varchar2  
);  
/  
  
create or replace type body color as  
    constructor function color(self in out color) return self as result is  
    begin  
        self.red := 255;  
        return;  
    end;  
    member function to_string return varchar2 is  
    begin  
        return '('|| self.red ||)';  
    end;  
end;  
/  
  
declare  
    v_line_color color;  
begin  
    v_line_color:=color();  
    dbms_output.put_line(v_line_color.to_string());  
end;  
/
```

10.3 集合类型在PL/SQL中应用

在PLSQL中，8s支持两种集合类型，可变数组和嵌套表。



元素	描述	限制	语法
typename	自定义类型的名称	符合8s名称定义规则	标识符

10.3.1 嵌套表类型的声明和使用

Nested_Table_Type_Spec:=



参数说明：

元素	描述	限制	语法
datatype	数据类型	基础数据类型以及自定义数据类型	标识符

例如：在下面应用场景中使用嵌套表类型。

```
--在PLSQL中定义并初始化嵌套表类型
declare
  type mytype is table of varchar(20);
  c2 mytype;
begin
  c2:=mytype('SMTIH','ALIEN','SCOTT','TOM');
  dbms_output.put_line(c2(2));
end;
/

ALIEN

PL/SQL procedure successfully completed.

--在PLSQL中定义多维数组
DECLARE
  TYPE tb1 IS TABLE OF VARCHAR2(20);
  vtb1 tb1 := tb1('one', 'three');
>
  TYPE ntb1 IS TABLE OF tb1;
  vntb1 ntb1 := ntb1(vtb1);
>
  TYPE tv1 IS VARRAY(10) OF INTEGER;
  TYPE ntb2 IS TABLE OF tv1;
  vntb2 ntb2 := ntb2(tv1(3,5), tv1(5,7,3));
>
```

```
BEGIN
    dbms_output.put_line('result:'|| vntb1(1)(1));
dbms_output.put_line('result:'|| vntb2.count);
END;
> /

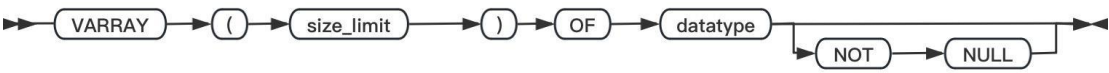
PL/SQL procedure successfully completed.

result:one
result:2
```

- 说明及限制:
- ORACLE模式下运行
 - 类型不支持serial、serial8、bigserial数据类型

10.3.2 可变数组的声明和使用

Varay_Type_Spec:=



参数说明:

元素	描述	限制	语法
size_limit	最大元素限制	整数值	数值
datatype	数据类型	基础数据类型以及自定义数据类型	标识符

例如：在下面应用场景中使用数组类型。

```
declare
    type mytype is varray(10) of varchar(20);
    c2 mytype;
begin
    c2:=mytype('SMTIH','ALIEN','SCOTT','TOM');
    dbms_output.put_line(c2(2));
end;
> /

PL/SQL procedure successfully completed.

ALIEN
```

说明及限制:

- ORACLE模式下运行
- 类型不支持serial、serial8、bigserial数据类型

11 自治事务

11.1 自治事务的创建

自治事务是由另一个事务（主事务）启动的独立事务，它在执行SQL操作并提交或回滚，而无需提交或回滚主事务。并且其还具有如下特点：

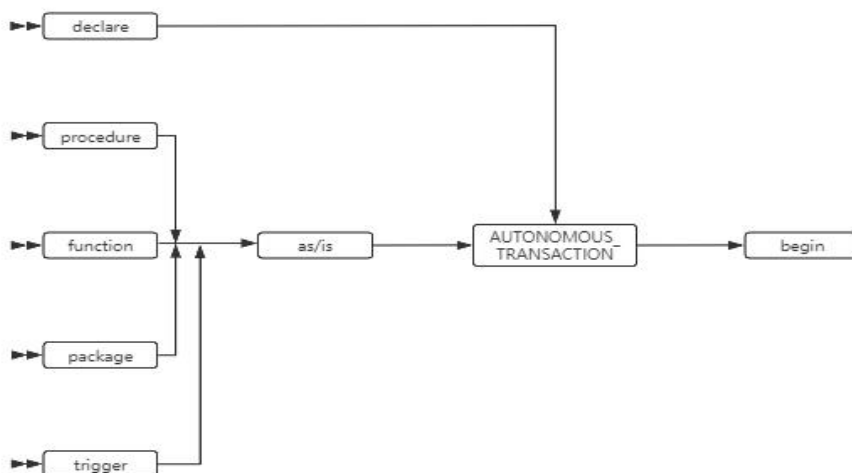
自治事务不与主事务共享事务资源（如锁）。

自治事务不依赖于主事务。例如，如果主事务回滚，则嵌套事务会回滚，但自治事务不会回滚。

自治事务提交的更改会立即对其他事务可见。

自治事务的异常部分属于外部事务，自治事务中引发的异常会导致事务级回滚，而非语句级回滚。

语法



11.2 自治事务的使用

- ORACLE模式下运行。
- 声明部分将例程标记为自治事务。
- 应用场景包括：存储过程、函数、包、触发器。
- 不支持重复声明自治事务。
- 不支持在嵌套块内声明自治事务。

- 当进入自治例程的可执行部分时，主事务暂停。回退自治例程后，主事务将恢复。
- COMMIT并结束活动的自治事务，但例程未执行结束。
- 保存点的作用域是在其中定义它的事务。事实上，主事务和自治事务可以使用相同的保存点名称。
- 自治例程依然属于当前会话，如时间格式、序列等以会话粒度使用的功能。自治例程与外部保持一致。
- 会话级隔离级别设置，自治事务设置后影响外部主事务事务属性，与其共享。
- 事务级隔离级别或属性设置，自治事务设置仅影响本事务，不影响外部主事务。亦或是自治例程内commit后由下一个语句开启的新事务，同样不受影响。

示例

```
create table employees(employee_id varchar(64),amount int,salary int);
insert into employees values(100,100,1000);
set environment sqlmode 'oracle';
```

创建plsql匿名块

```
DECLARE

    PRAGMA AUTONOMOUS_TRANSACTION;

    emp_id NUMBER(6)      := 100;

    amount NUMBER(6,2) := 200;

BEGIN

    UPDATE employees SET salary = salary - amount
    WHERE employee_id = emp_id;

    COMMIT;

END;

/
```

创建存储过程

```
CREATE OR REPLACE PROCEDURE lower_salary
    (emp_id NUMBER, amount NUMBER)
AUTHID DEFINER AS
    PRAGMA AUTONOMOUS_TRANSACTION;

BEGIN

    UPDATE employees
    SET salary = salary - amount
    WHERE employee_id = emp_id;

    COMMIT;

END lower_salary;

/
```

创建包

```
CREATE OR REPLACE PACKAGE emp_actions AUTHID DEFINER AS
    FUNCTION raise_salary (emp_id NUMBER, sal_raise NUMBER)
    RETURN NUMBER;
```



```
END emp_actions;
/
CREATE OR REPLACE PACKAGE BODY emp_actions AS
    FUNCTION raise_salary (emp_id NUMBER, sal_raise NUMBER)
    RETURN NUMBER IS
        PRAGMA AUTONOMOUS_TRANSACTION;
        new_sal NUMBER(8, 2);
    BEGIN
        UPDATE employees SET salary =
            salary + sal_raise WHERE employee_id = emp_id;
        COMMIT;
        SELECT salary INTO new_sal FROM employees
            WHERE employee_id = emp_id;
        RETURN new_sal;
    END raise_salary;
END emp_actions;
/
```

创建触发器

```
CREATE TABLE log(
log_id NUMBER(6), up_date DATE, new_sal NUMBER(8, 2), old_sal  NUMBER(8, 2));
CREATE TABLE emp(empno varchar(32), sal int);
CREATE OR REPLACE TRIGGER log_sal
    BEFORE UPDATE OF sal ON emp FOR EACH ROW
DECLARE
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO log (
        log_id,
        up_date,
        new_sal,
        old_sal
    )
    VALUES (
        :old.empno,
        SYSDATE,
        :new.sal,
        :old.sal
    );
    --COMMIT;
```

END;

/

12 错误处理

PL/SQL运行时的错误可能由设计错误、编码错误、硬件引起失败，还有很多其他的原因。你不能预料到所有可能的错误。但你可以通过异常处理来让你的程序在发生错误后还能继续执行。

12.1 PL/SQL运行时的错误处理概述

在PL/SQL中，错误条件称为异常。异常可以是内部定义的（由运行时系统）或用户定义的。内部定义的异常示例有no_data_found（没有找到数据）和zero_divide（除以0）。

您可以在任何PL/SQL块、子程序的声明部分定义自己的异常。例如，您可以定义一个名为“资金不足”的异常来标记透支的银行帐户。必须为用户定义的异常命名。

发生错误时，会引发异常。也就是说，正常的执行停止，控制权转移到PL/SQL块或子程序的异常处理部分。运行时系统隐式（自动）引发内部异常。用户定义的异常必须由RAISE或RAISE_APPLICATION_ERROR抛出。

要处理引发的异常，可以编写称为异常处理程序的单独例程。异常处理程序运行后，当前块停止执行，上层块继续执行下一条语句。如果没有上层块，则返回的上层存储过程，直到返回主机执行环境。

例如除0异常：

```
DROP TABLE t2; -- t2表用来保存调试信息

CREATE TABLE t2(c1 VARCHAR(200));

CREATE OR REPLACE PROCEDURE pr_divide_zero (sp1 INT, sp2 INT) IS
    res float;
BEGIN
    res:=sp1/sp2; --这一句发生除0错误

    EXCEPTION

        WHEN ZERO_DIVIDE THEN --这一句截获刚刚发生的除0错误

            INSERT INTO t2 VALUES('zero_divide. '); --记录调试信息到t2

END;

/

CALL pr_divide_zero (1,0); --调用存储过程pr_divide_zero
```

```
SELECT * FROM t2; --通过t2查看调试信息
```

程序执行的流程如下：

- 调用存储过程pr_divide_zero(1,0)
- 在pr_divide_zero内发生除0错误, 此时自动抛出异常zero_divide
- 截获刚刚抛出的zero_divide异常
- 记录调试信息到t2中,然后回退出当前执行块,返回上层块的下一句
- 通过t2查看调试信息

12.2 预定义异常

如果PL/SQL程序违反数据库规则或超过系统相关限制，则会自动引发内部异常。
PL/SQL将一些常见的错误预定义为异常。例如，如果SELECT-INTO语句不返回行，PL/SQL将引发预定义的异常NO-DATA。

现在支持的预定义异常如下表：

异常名	抛出异常的情况
no_data_found	select into语句一行也没有返回
too_many_rows	select into语句返回多于一行数据
zero_divide	尝试除以0
dup_val_on_index	在有唯一索引的列上尝试存入重复值
invalid_number	尝试将字符串转化为数字,但这个字符串不是合法的数字

12.3 自定义异常

PL/SQL允许您定义自己的异常。与预定义异常不同，必须声明用户定义的异常，然后才能使用RAISE语句或RAISE_APPLICATION_ERROR显式抛出异常。其中RAISE_APPLICATION_ERROR允许您将错误消息与用户定义的异常关联起来。

★声明异常

异常只能在存储过程、PL/SQL块的声明部分声明。通过引入异常的名称和关键字exception来声明异常。

在存储过程的声明部分声明异常：

```
CREATE TABLE t2(c1 VARCHAR(200));  
CREATE OR REPLACE PROCEDURE p_6_3_1_1 (age INT) IS
```

```
illegal_age EXCEPTION;    -- 声明异常

BEGIN
  IF age < 0 THEN
    RAISE illegal_age; -- raise an exception that you defined
  END IF;
  EXCEPTION
    WHEN illegal_age THEN
      INSERT INTO t2 VALUES('out_of_stock');  -- 插入调试信息
END;
/
```

在块的声明部分声明异常:

```
CREATE TABLE t2(c1 VARCHAR(200));
CREATE OR REPLACE PROCEDURE p_6_3_1_2 IS
BEGIN
  DECLARE    --sub-block begins
    subblock_exception EXCEPTION;  -- 在块的声明区声明异常
  BEGIN
    RAISE subblock_exception;
  EXCEPTION
    WHEN subblock_exception THEN
      INSERT INTO t2 VALUES('catch subblock_exception');
  END;
end;
/
```

异常和变量声明类似。但请记住，异常是错误条件，而不是数据项。与变量不同的是，异常不能出现在赋值语句或SQL语句中。和变量相同的是，作用域规则。

★PL/SQL异常的作用域规则

块之间存在两种关系：父子关系和兄弟关系。

多层父子关系构成直系祖先关系。

```
CREATE OR REPLACE PROCEDURE p_6_3_2_1 IS
BEGIN    -- 块 1

  BEGIN    -- 块1.1
    BEGIN  -- 块1.1.1

    END;
  BEGIN;
```

```
BEGIN    -- 块1.2

BEGIN;

END;
```

块1是存储过程的根块。块1.1，1.2是块1的子块。块1.1和1.2之间是兄弟块。块1.1.1是块1.1的子块。块1是块1.1.1的直系祖先块。

异常作用域规则：

- 不能在同一块中声明两次同名异常
- 一个块中可以声明和他的父块或直系祖先块相同的异常
- 在块中声明的异常被认为是该块的本地异常，而对该块所有子块都是全局异常
- 块内只能引用局部或全局异常，父块不能引用子块中声明的异常
- 如果在子块中重新声明全局异常，则以本地声明为准，此时子块不能引用全局异常

举例说明：

不能在同一块中声明两次异常。如：

```
DROP PROCEDURE p_6_3_2_2;
CREATE OR REPLACE PROCEDURE p_6_3_2_2 IS
    a EXCEPTION;  -- 第一次声明异常a
    a EXCEPTION;  -- 第二次声明异常a, 重复声明
BEGIN
    RAISE a;
    EXCEPTION
        WHEN THEN
            NULL;
END;
/
```

一个块中可以声明和他的父块或直系祖先块相同的异常：

```
CREATE TABLE t2(c1 VARCHAR(200));
DROP PROCEDURE sp1;
CREATE OR REPLACE PROCEDURE p_6_3_2_3 IS
    a EXCEPTION;  -- 存储过程的根块声明了异常a
BEGIN
    DECLARE
        a EXCEPTION;  --在子块中也声明了异常a
    BEGIN
```

```
NULL;  
END;  
END;  
/  
CALL sp1();  
SELECT * FROM t2;
```

在块中声明的异常被认为是该块的本地异常，而对该块所有子块都是全局异常。

```
CREATE OR REPLACE PROCEDURE p_6_3_2_4 IS  
  a EXCEPTION; -- 存储过程的根块声明了异常a，对于存储过程a是本地异常  
BEGIN  
  -- 这里a是本地异常  
  BEGIN  
    -- 这里a 是全局异常  
    BEGIN  
      -- 这里a也是全局异常  
      END;  
    END;  
  END;  
END;
```

块内只能引用局部或全局异常，父块不能引用子块中声明的异常。

```
CREATE OR REPLACE PROCEDURE p_6_3_2_5 IS  
  a EXCEPTION;  
BEGIN -- 块 1  
  DECLARE  
    b EXCEPTION;  
  BEGIN -- 块1.1  
    -- 这里b是本地异常，a是全局异常。  
  END;  
  --这里a是本地异常，b不可见。  
END;
```

如果在子块中重新声明全局异常，则以本地声明为准。此时子块不能引用全局异常。

★定义自己的错误消息

RAISE_APPLICATION_ERROR过程允许您从存储的子程序发出用户定义的错误消息。这样，您就可以向应用程序报告错误并避免返回未处理的异常。

要调用RAISE_APPLICATION_ERROR，请使用以下语法：

```
raise_application_error(error_number,message [, {TRUE | FALSE}]);
```

其中，`error_number`是-20000..-20999范围内的负整数，而`message`是一个最长2048字节的字符串。

应用程序只能从正在执行的存储子程序（或方法）调用`raise_application_error`。调用时，`raise_application_error`结束子程序并向应用程序返回用户定义的错误号和消息。错误号和消息可以像其他异常一样被捕获。

示例如下：

```
CREATE TABLE t2(c1 VARCHAR(200));
CREATE TABLE user_tables(name VARCHAR(50));
INSERT INTO user_tables VALUES('a');
INSERT INTO user_tables VALUES('b');
INSERT INTO user_tables VALUES('c');

CREATE OR REPLACE PROCEDURE p_6_3_3 IS
    num_tables NUMBER;
BEGIN
    SELECT COUNT(*) INTO num_tables FROM user_tables;
    IF num_tables < 10 THEN
        RAISE_APPLICATION_ERROR(-20101, 'expecting at least 10 tables'); -- 抛出异常-20101
    END IF;
END;
/
DROP PROCEDURE sp2;
CREATE OR REPLACE PROCEDURE sp2 IS
    null_salary EXCEPTION;
    PRAGMA EXCEPTION_INIT(null_salary, -20101); -- 将异常名null_salary和-20101关联
begin
    sp1();
    EXCEPTION
        WHEN null_salary THEN -- 截获null_salary异常
            INSERT INTO t2 VALUES('null_salary happen.');
```

此技术允许调用应用程序处理特定异常处理程序中的错误条件。

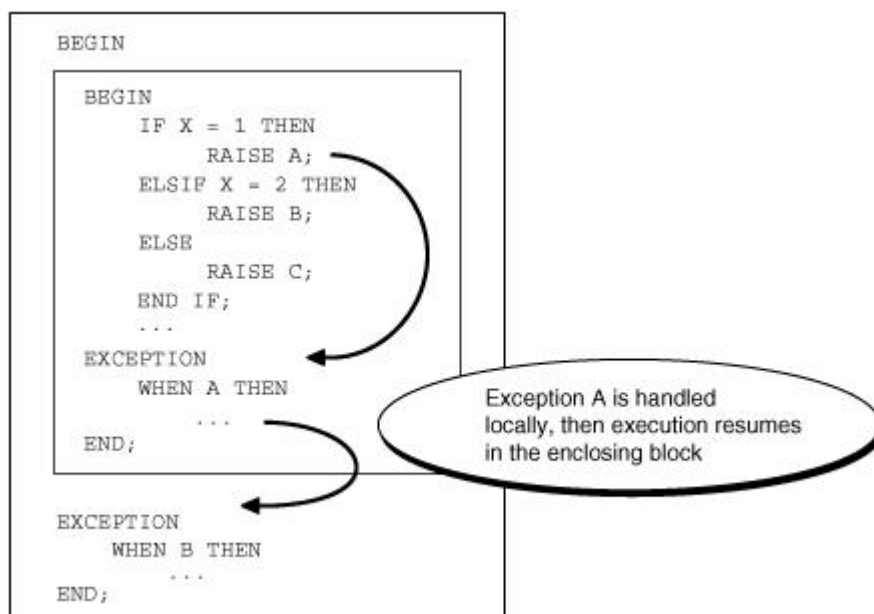
12.4 如何抛出异常

内部异常是由运行时系统隐式抛出的。自定义异常必须显式抛出，包括RAISE语句或调用过程RAISE_APPLICATION_ERROR。有关抛出异常的示例，请参见其它章节。

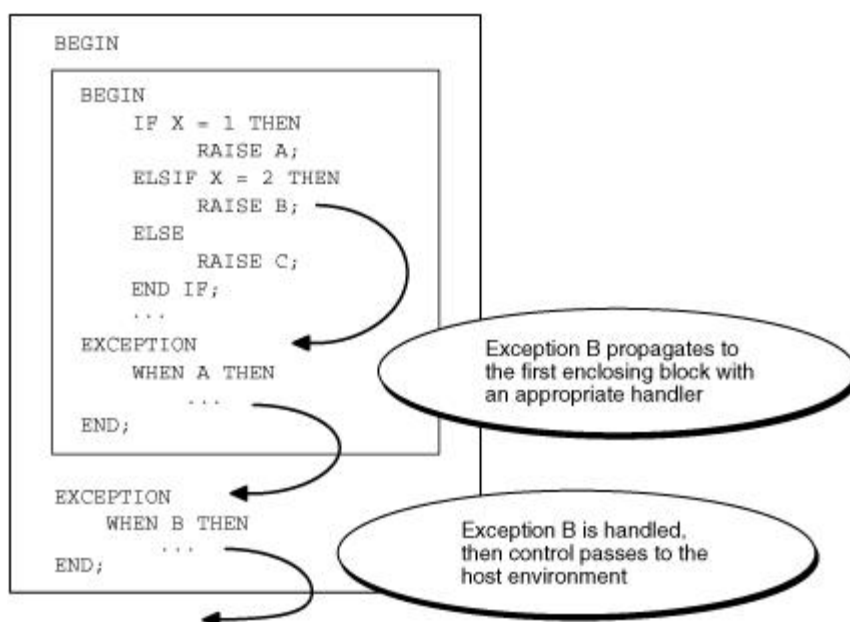
12.5 异常如何传播

当引发异常时，如果PL/SQL在当前块或子程序中找不到该异常的处理程序，则异常会向上传播。也就是说，异常在层层父块中复制自身，直到找到一个处理程序为止。如果最后也找不到处理程序，PL/SQL将向主机环境返回一个未处理的异常错误。

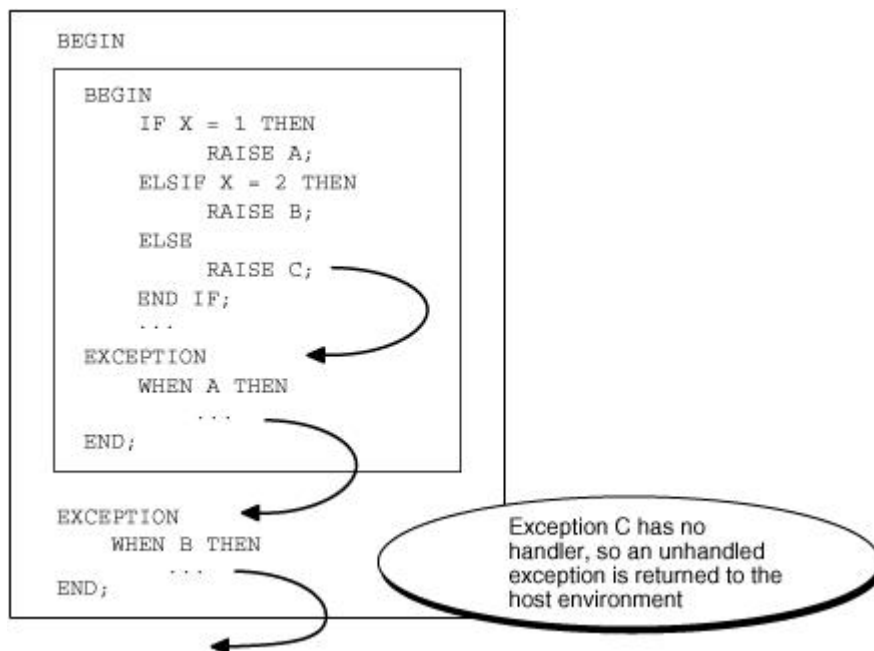
下图说明了基本的传播规则。



子块中抛出的异常A在子块中就被截获并处理了，此时控制权交给了父块。



子块中抛出的异常 B 在父块中被截获并处理了，此时控制权交给了外部环境。



子块中抛出的异常 C 没有被截获，因此返回给执行环境一个错误：未处理的异常。

异常可以传播到其作用域之外，也就是说，在声明它的块之外。 例如：

```

CREATE OR REPLACE PROCEDURE p_6_5 IS
  past_due EXCEPTION;--子块中声明的异常

  due_date DATE := trunc(SYSDATE) - 1;
  todays_date DATE := trunc(SYSDATE);
BEGIN
  IF due_date < todays_date THEN
    RAISE past_due;--抛出异常past_due
  END IF;
  ----- 子块结束

EXCEPTION
  WHEN OTHERS THEN-- 处理异常

    ROLLBACK;
END;
/

```

由于声明异常 `past_due` 的块没有处理异常，因此异常将传播到父块。但是，父块不能引用 `past_due` 的名称，因为它声明的范围不再存在。一旦异常名称不再存在，只有 `OTHERS` 处理程序才能截获这个异常。

12.6 重新抛出异常

有时，需要重新抛出异常，即在当前块处理异常，然后将其传递给父块。例如，您可能希望回滚当前块中的事务，然后在封闭块中记录错误。

若要重新抛出异常，请使用不带异常名称的RAISE语句，该语句仅允许在异常处理程序中使用。例如：

```
CREATE TABLE t2(c1 VARCHAR(200));
CREATE OR REPLACE PROCEDURE p_6_6 IS
    salary_too_high EXCEPTION;
    current_salary NUMBER := 20000;
    max_salary NUMBER := 10000;
    erroneous_salary NUMBER;
BEGIN
    BEGIN ----- sub-block begins
        IF current_salary > max_salary THEN
            RAISE salary_too_high; -- raise the exception
        END IF;
        EXCEPTION
            WHEN salary_too_high THEN--子块中处理异常
                -- first step in handling the error
                insert into t2 values('salary_too_high in sub-block.');
```

RAISE; --重新抛出异常

```
        END; ----- sub-block ends
    EXCEPTION
        WHEN salary_too_high THEN--父块中处理异常
            -- handle the error more thoroughly
            insert into t2 values('salary_too_high in top-block.');
```

erroneous_salary := current_salary;

current_salary := max_salary;

```
END;
/
```

12.7 处理抛出的异常

当引发异常时，PL/SQL块或子程序的正常执行将停止，并将控制传输到其异常处理程序，其格式如下：

```
EXCEPTION

    WHEN exception1 THEN -- 处理异常exception1

        处理语句序列1

    WHEN exception2 THEN -- 处理另一个异常exception2
```

```
    处理语句序列2
...
    WHEN OTHERS THEN -- 处理所有其它异常
        处理语句序列3
END;
```

要捕获引发的异常，请编写异常处理程序。每个处理程序都包含一个**WHEN**子句，该子句指定一个异常，然后是引发该异常时要执行的一系列语句。这些语句执行完之后，执行控制权不返回抛出异常的位置。换言之，您不能在异常中断的地方继续处理。

可选的**OTHERS**异常处理程序（始终是块或子程序中的最后一个处理程序）充当所有未明确命名的异常的处理程序。因此，一个块或子程序只能有一个**OTHERS**处理程序。使用**OTHERS**处理程序可以保证不会出现未处理的异常。

如果希望两个或多个异常执行相同的语句序列，请在**WHEN**子句中列出异常名称，并用关键字**OR**分隔它们，如下所示：

```
EXCEPTION
    WHEN over_limit OR under_limit OR VALUE_ERROR THEN
        -- 处理这些异常
```

如果引发列表中的任何异常，则执行相关的语句序列。关键字**OTHERS**不能出现在异常名称列表中；它必须自己出现。可以有任意数量的异常处理程序，每个处理程序都可以将异常列表与一系列语句相关联。但是，异常名称只能在**PL/SQL**块或子程序的异常处理部分出现一次。

PL/SQL变量的通常作用域规则适用，因此您可以在异常处理程序中引用本地和全局变量。但是，当在**cursor FOR**循环中引发异常时，在调用处理程序之前会隐式关闭游标。因此，显式游标属性的值在处理程序中不可用。

★在声明部分抛出的异常

对于**oracle**而言，当前块中的处理程序无法捕获引发的异常，因为声明中引发的异常会立即传播到封闭块。

咱们的产品，可以在当前块中截获声明部分抛出的异常。例如：

```
DROP TABLE t2;
CREATE TABLE t2(c1 VARCHAR(200));
DROP PROCEDURE sp1;
CREATE OR REPLACE PROCEDURE p_6_7_1 (X INT) IS
    A EXCEPTION;
begin
```

```
DECLARE-- 子块开始

    i int := 10/X;-- 声明区发生异常（除以0）

BEGIN

    NULL;

    EXCEPTION

        WHEN OTHERS THEN

            INSERT INTO t2 VALUES('咱们的产品可以在这里截获。');

END;-- 子块结束

EXCEPTION

    WHEN OTHERS THEN

        INSERT INTO t2 VALUES('oracle在这里截获');

end;

/

CALL sp1(0);

SELECT * FROM t2;
```

★处理在异常处理程序中抛出的异常

当一个异常A在一个异常处理程序中抛出时，同一个异常处理程序无法控制异常A。此时，异常A会传播到父块，从那儿，才能继续截获异常A。比如：

```
EXCEPTION

    WHEN INVALID_NUMBER THEN

        INSERT INTO ... -- 可能跑出异常 DUP_VAL_ON_INDEX

        WHEN DUP_VAL_ON_INDEX THEN -- 不能截获上一句抛出的DUP_VAAAL_ON_INDEX

END;
```

★GOTO语句和异常处理

GOTO语句可以从异常处理程序跳转到父块。例如：

```
CREATE TABLE t2(c1 VARCHAR(200));

CREATE OR REPLACE PROCEDURE p_6_7_3 (X INT) IS

begin

    <<label1>>

    DECLARE

        -- Raises an error:

        i int := 10;

    BEGIN

        i := 10 / x;

        INSERT INTO t2 VALUES('now x is not 0');
```

```
EXCEPTION
  WHEN zero_divide THEN
    INSERT INTO t2 VALUES('in sub-block, zero_divide');
    x := 10;
    GOTO label1;
  END;
end;
/
CALL sp1(0);
SELECT * FROM t2;
```

The logo for GBASE, featuring the word "GBASE" in a bold, red, sans-serif font, followed by a registered trademark symbol (®). To the left of the text is a red square.

南大通用数据技术股份有限公司
General Data Technology Co., Ltd.



微信二维码

